第 3 章 轨迹大数据治理

随着移动互联网的发展，轨迹数据在规模迅速扩大的同时，其采集来源、生成环境和记录形态也日益复杂，原始数据中的质量问题也随之增多[1]。例如，卫星信号遮挡、定位设备误差和通信延迟可能造成轨迹点漂移、时间戳错乱或轨迹片段缺失[2]；高频采样会带来大量冗余点，增加存储和计算压力；多源轨迹数据之间还可能存在坐标系统不统一、字段格式不一致和采样频率差异等问题。如果不对这些问题进行系统治理，后续的轨迹存储、轨迹查询、轨迹分析和挖掘结果都可能受到严重影响。

作为轨迹数据管理与分析流程中的基础环节，轨迹大数据治理是指通过一系列标准化、自动化的技术手段，对原始轨迹进行规范化的过程。本章围绕轨迹数据治理的基本概念、核心任务和典型方法展开，重点介绍轨迹质量评估、轨迹清洗、轨迹压缩和轨迹恢复等内容，为后续轨迹分析与应用提供高质量的数据基础。

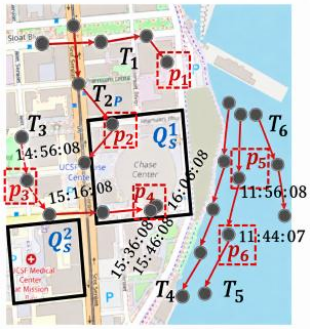
3.1 轨迹大数据治理概述

轨迹数据治理（Trajectory Data Governance）是对多源轨迹记录进行质量控制、标准化、组织与管理的过程。它通过规范数据组织、识别和处理质量问题，提高轨迹数据的一致性与可用性，为城市计算、智能交通等应用提供数据支持。

作为一项综合性工作，轨迹数据治理涵盖的任务十分广泛，既包括数据质量提升，也包括数据组织、结构优化和数据增强等多个方面。本章主要选取轨迹质量评估（Trajectory Data Quality Assessment）、轨迹清洗（Trajectory Cleaning）、轨迹压缩（Trajectory Compression）和轨迹恢复（Trajectory Recovery）四类具有代表性的基础任务进行介绍。它们分别对应轨迹数据治理过程中的质量诊断、错误修正、冗余削减和缺失补全等环节，是后续轨迹分析与应用的重要基础。

轨迹质量评估是轨迹数据治理中的前置环节，主要用于判断原始轨迹数据是否可靠、完整和可用。如图 3.1 所示，受采集环境与设备限制，原始轨迹常常会出现大量的质量缺陷，例如违反时间戳顺序（T_5）、轨迹点丢失（T_3）、轨迹漂移（T_2）等。通过轨迹质量评估，可以及时发现轨迹数据中的缺失、异常、漂移、采样不均和时空不连续等质量问题，并为后

续的其他轨迹治理任务提供依据。因此，轨迹质量评估不仅是发现问题的过程，也是保障后续轨迹分析结果可靠性的基础。



Dimension	Trajectory	Description
Validity	T_5	*p_5,p_6 violate the timestamp monotonicity*
Completeness	T_3	*p_3 is a missing point*
Consistency	T_1	*p_1 is positioned incorrectly inside a building*
	T_2	*p_2 suffers from a location shift*
	T_3	*The vehicle should not stay on the road (p_4)*
	T_6	*T_6 deviates from the course of T_4,T_5*

图 3.1 轨迹质量评估任务的示意图（改绘自[3]）

轨迹清洗主要用于处理原始轨迹数据中存在的噪声点、异常点、漂移点等问题，其目标是识别并修正错误观测，使轨迹数据更加可靠和稳定。如图 3.2 所示，相比于存在不合理局部跳变的原始轨迹（黑色曲线），经异常校正的修复轨迹（蓝色曲线）有效保留了原有变化趋势，且更加连续、稳定；红色曲线表示了平滑处理的结果，用于与异常校正结果对比。在实际操作中，通常需结合距离、速度、加速度等时空与运动特征进行综合判断，对偏离正常规律的点位执行删除、修正或平滑操作。清洗结果应通过误差或下游任务指标验证，避免平滑处理掩盖真实运动变化。

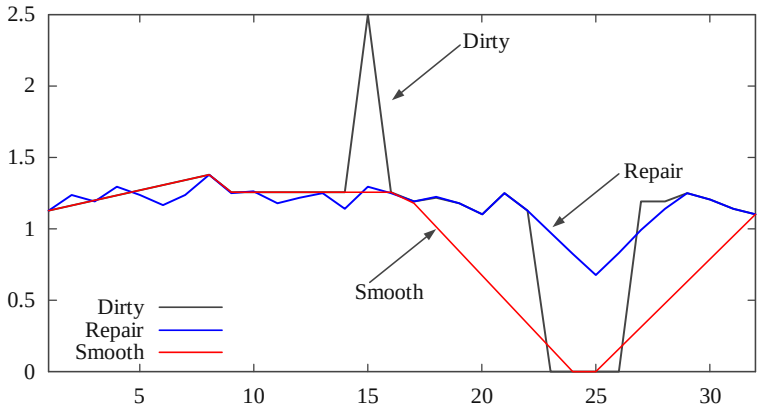


图 3.2 轨迹清洗任务的示意图

轨迹压缩旨在保留原始轨迹整体形态与语义特征的前提下，通过剔除冗余点来缩减数据规模。如图 3.3 所示，压缩过程并非简单的等距抽样，而是基于各点对轨迹形状的贡献度进行动态取舍：首尾节点（P_1、P_6）及体现方向变化的转折点（P_3）等关键信息通常会被保留；而对整体形态影响微弱的中间点（P_2、P_5）则作为冗余被删除。因此，轨迹压缩的核心是在降低存储与计算成本的同时，最大程度地避免关键运动特征（如转折、停留）的丢失，从而实现数据规模与信息保真度之间的平衡。

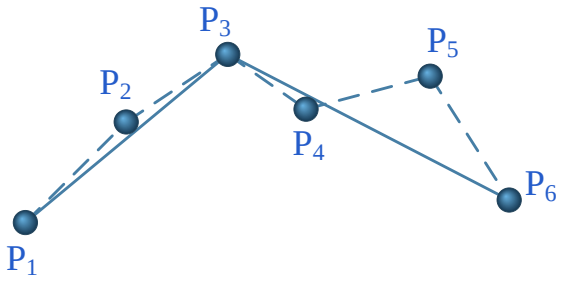


图 3.3 轨迹压缩任务的示意图

轨迹恢复旨在解决由于采样稀疏或信号中断导致的片段缺失问题，通过已有观测信息对不完整的行驶过程进行补全与重建。如图 3.4 所示，受限于实际观测条件（如摄像头盲区），车辆在路网中往往仅记录下离散的已观测点（红色节点），而未被捕捉的盲区则构成了轨迹断层（灰色节点）。轨迹恢复的核心，便是综合已观测点的时空关联、运动规律、路网拓扑及历史行为模式，借助插值、地图匹配或机器学习模型等方法，对缺失片段进行合理推断。通过这一环节，原本离散的中断轨迹得以还原为连续、完整的物理移动过程，提高了数据的完整性。

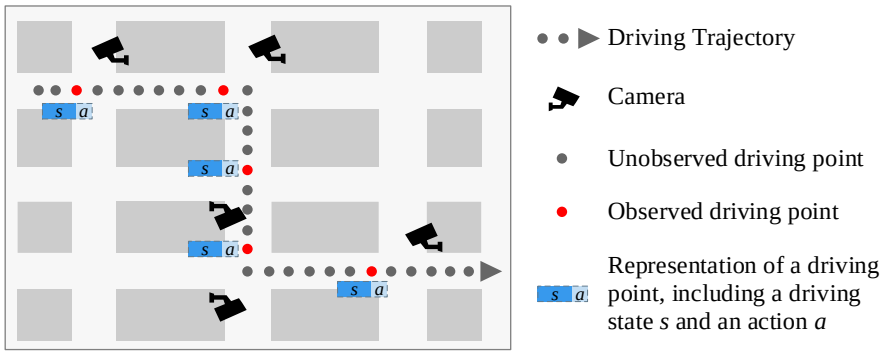


图 3.4 轨迹恢复任务的示意图

上述四类任务并不能涵盖轨迹数据治理的全部内容。在实际系统中，轨迹数据治理还可能涉及轨迹标准化、坐标转换、轨迹分段、地图匹配、语义标注、隐私保护和数据融合等任务。本章选取质量评估、清洗、压缩和恢复作为重点，是因为它们直接面向原始轨迹数据中最常见的质量与结构问题，能够较好体现轨迹大数据治理的基本思想和关键技术路径。

3.2 轨迹质量评估

在轨迹数据管理生命周期中，轨迹质量评估是保障后续分析、挖掘与智能决策可靠性的基础环节[4]。它通过识别质量问题、构建评价指标并量化评估结果，判断轨迹数据的可用性，为后续清洗、恢复及下游应用提供依据。

本节从通用数据质量评估和轨迹专用质量评估两个层面介绍相关方法。通用评估提供完整性、一致性、有效性等基础评价思路，并可根据是否考虑下游任务，区分任务无关评估与任务感知评估；轨迹专用评估进一步引入时空关系、运动规律、道路约束及覆盖差异。下文首先界定轨迹质量评估的对象与常见问题，再介绍通用方法及轨迹专用框架。

3.2.1. 轨迹质量评估概述

轨迹质量评估面向采样点、单条轨迹及轨迹数据集，依据采集条件、检查规则或任务要求，识别并量化数据缺陷。评估结果通常包括质量指标、问题记录及其位置，为后续清洗、恢复和数据筛选提供依据。轨迹质量问题可能产生于定位、采样和传输等环节，常见影响因素包括以下三个方面。

1. 定位系统存在测量误差：GPS、北斗卫星导航系统及移动基站定位均存在精度限制。在城市峡谷、高楼密集区等环境中，信号遮挡和多径效应会降低定位精度；在隧道、地下空间等区域，卫星定位还可能无法获得有效位置，使轨迹出现漂移或缺失[5]。

2. 采样频率受到资源约束：移动设备在采集轨迹数据时，需要在定位精度、存储容量和通信成本之间进行权衡。为降低能耗或减少通信开销，许多设备会采用较低的采样率。低采样率虽然可以节约资源，但会导致真实移动过程中的细节信息丢失，进而造成稀疏轨迹[6]。

3. 网络传输过程发生故障：轨迹数据通常需要从终端设备上传至服务器或云端。在网络信号不稳定、通信中断或数据传输延迟的情况下，部分轨迹点可能无法及时上传，甚至永久丢失，从而影响轨迹数据的连续性和实时性[7]。

上述物理与技术层面的限制，会在轨迹数据的逻辑结构中表现为多种时空质量问题。根据其对轨迹时空特征的破坏形式，常见问题可归纳为以下几类。

1. 空间与拓扑错误：定位漂移或坐标误差可能使轨迹点远离实际活动区域。判断时需考虑定位误差容限、路网完整性及桥梁、隧道和停车场等场景，不能仅凭点位位于水域或建筑物范围内就认定为错误。

2. 时间序列异常：同一移动对象的轨迹点通常应按照时间顺序排列，并满足时间戳单调递增的要求。如果由于设备时钟异常、数据乱序或多源数据合并错误，出现后续轨迹点时间戳早于前序轨迹点的情况，就会破坏轨迹的时间一致性。

3. 连续性与完整性缺失：轨迹数据中可能存在采样间隔异常过长、部分时间段无记录、关键字段缺失等问题，导致轨迹在时间或空间维度上出现不连续现象。例如，连续两个轨迹点之间的时间间隔过大，可能说明中间存在大量未采集的轨迹点，从而造成轨迹片段缺失。

4. 运动学逻辑冲突：相邻点的位移、速度、加速度或方向变化超过合理范围时，可将相关观测标记为异常候选。例如，由相邻两点计算出的车辆速度超过 300 km/h。

5. 数据分布不均衡：轨迹数据在空间和时间上的覆盖往往并不均匀。例如，城市中心区域轨迹点密集，而郊区或偏远区域数据稀疏；工作日高峰时段数据充分，而夜间或节假日数据不足。这种时空分布不均可能导致下游模型在数据充足区域表现较好，而在数据稀疏区域产生较大的预测偏差或较高的漏检率。

上述问题可能同时出现，同一异常记录也可能违反多项约束。评估时应分别记录触发的检查规则与对应位置，便于后续核查和处理。

3.2.2. 通用数据质量评估方法

通用数据质量评估方法可按是否考虑下游任务需求，分为任务无关质量评估（Task-agnostic Quality Assessment）[8,9]和任务感知质量评估（Task-aware Quality Assessment）[10]。前者关注数据本身是否准确、完整、一致和合理；后者关注质量问题对特定任务的影响。二者分别回答“这批数据本身是否可靠？”和“这批数据是否适合当前任务？”。

（1） 任务无关的数据质量评估

该类方法不依赖具体下游应用，而是从轨迹数据自身的时空结构、记录状态和属性约束出发，对数据质量进行普适性判断。无论轨迹数据后续用于地图匹配、交通预测、行为识别还是出行模式挖掘，都需要首先满足基本的数据可靠性要求。因此，任务无关评估可以视为轨迹质量评估的基础层。早期任务无关的轨迹质量评估主要依赖人工规则和物理约束，通常围绕空间质量、时间质量以及属性与逻辑质量展开，旨在衡量轨迹数据的准确性、完整性、一致性和逻辑合理性。

1. 空间质量评估：空间质量主要用于衡量轨迹点坐标是否接近真实物理位置，以及轨迹点的空间分布是否合理。在缺乏真实位置基准的情况下，通常可以通过轨迹点是否落在合理活动区域内进行间接判断。例如，车辆轨迹点若大量出现在水域、建筑物内部或道路网络之外，往往说明数据中可能存在定位漂移或空间误差。同时，移动对象的运动应具有空间连续性，相邻轨迹点之间的距离不应超过其在对应时间间隔内可能达到的物理运动范围。若相邻两点距离过大，形成异常的“空间跳跃”（Spatial Jump），则通常可能存在空间质量问题。

2. 时间质量评估：时间质量主要关注轨迹点时间戳是否正确、有序和连续。作为时空序列，同一轨迹内部的时间戳应满足单调递增关系，即$t_{i+1} > t_i$。因此，时间质量评估需要识别乱序记录、重复时间戳、时间回退等异常情况。这类问题通常来源于设备时钟不同步、网

络传输延迟或多源数据合并错误。对于固定采样频率的数据，若相邻点的时间间隔明显增大，可能存在信号丢失或数据缺失；对于事件触发或自适应采样，应结合采集策略判断。

3. 属性与逻辑质量评估：属性与逻辑质量主要检查轨迹附属字段是否完整、合理，并判断其是否符合基本物理和地理逻辑。首先，需要检查速度、方向角等关键字段是否缺失或异常。其次，需要验证属性与轨迹坐标之间是否一致，例如记录速度应与相邻轨迹点计算得到的运动速度大体一致，方向角变化应与轨迹整体走向相符。最后，还需要结合地理拓扑规则进行判断，例如车辆轨迹应符合道路连通关系和单行规则，高铁轨迹应依附于铁路网络等。通过这些检查，可以识别字段缺失、运动状态矛盾和拓扑约束违背等质量问题。

早期基于规则的评估方法能够识别部分明显的轨迹质量问题，具有易实现和可解释性强等优点。然而，这类方法往往是静态的、局部的和分散的，评估结果高度依赖人工经验和阈值设定。随着轨迹数据规模不断扩大、数据来源日益多样，仅依靠人工规则逐项检测已难以满足大规模轨迹数据质量管理需求。因此，研究者开始将分散的数据质量问题抽象为统一的指标体系，并进一步构建更加系统化、自动化和框架化的数据质量评估方法。下面将介绍一种代表性的任务无关质量评估方法，并分析其在轨迹数据场景中的应用方式。

TsQuality[11]是一种面向 Apache IoTDB 中大规模时间序列数据的质量评估系统。该系统将时间序列数据质量概括为完整性、一致性、及时性和有效性四个核心维度，并将相关指标实现为 IoTDB 函数或 Spark 算子，以支持大规模时间序列质量分析。

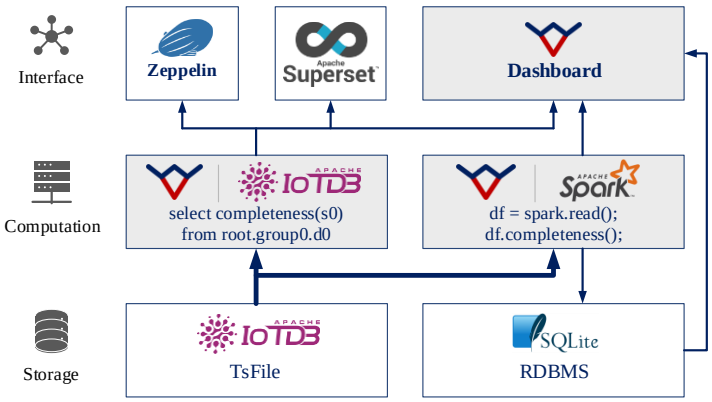


图 3.5 TsQuality 的框架图（改绘自[11]）

TsQuality 的整体框架如图 3.5 所示，采用“存储层—计算层—接口层”的三层架构。存储层以 Apache IoTDB 的 TsFile 作为时间序列数据的底层存储格式，同时使用 SQLite 保存质量统计结果。计算层提供两类质量评估方式：一类是将完整性、一致性、及时性和有效性等质量指标实现为 IoTDB 原生函数，使用户能够直接通过 SQL 语句查询某条时间序列或某一时间窗口内的数据质量；另一类是将这些指标实现为 Spark 算子，以支持大规模时间序列数

据的批量质量计算。接口层则面向用户提供可视化和交互分析能力，包括 IoTDB 原生 Dashboard、Apache Zeppelin 和 Apache Superset 等工具。

由于轨迹数据本质上也是一种按照时间顺序组织的序列数据，其采样点通常由时间戳、空间位置及相关属性共同构成，因此可以借鉴 TsQuality 的指标体系，对轨迹数据质量进行更加系统的评估。在轨迹数据场景中，完整性衡量预期采样点是否缺失；一致性检查重复记录及同一对象、同一时刻的位置冲突；及时性衡量轨迹点的上传或入库延迟，适用于调度、交通监测等实时任务；有效性检查经纬度范围、速度和加速度等预设约束。通过将分散的时间序列质量问题组织为一套统一的度量体系，避免了传统规则检测中指标零散、口径不统一的问题，使轨迹质量评估更加系统化和可解释。

（2）任务感知的数据质量评估

传统的任务无关质量评估旨在提供一种通用的数据诊断标准，但在实际分析中，这种试图用单一标准适配所有场景的模式往往暴露出明显的局限性。为了弥补这一不足，任务感知的质量评估应运而生。任务感知的质量评估不仅关注轨迹数据本身是否存在缺失、重复、漂移或逻辑冲突，还进一步考虑这些质量问题对具体下游任务的影响。其基本观点是：数据质量并不是完全绝对的概念。同一种质量问题在不同应用场景下可能具有不同的重要性；同一份轨迹数据对于某些任务可能已经足够可用，但对于另一些任务则可能无法满足要求。

例如，短时间的轨迹缺失对于起讫点统计的影响可能较为有限，因为该类任务主要关注出发点、到达点和总体流向。然而，对于轨迹预测、驾驶行为识别或交通冲突检测而言，短时间缺失可能破坏连续运动模式，使模型难以准确捕捉速度、加速度和方向变化。又如，速度字段缺失对于区域访问频次统计影响较小，但对于交通状态识别、拥堵传播分析和危险驾驶检测则可能造成显著影响。因此，任务感知评估并不只是判断“数据是否存在问题”，而是进一步判断“哪些问题会影响当前任务，以及影响程度有多大”。

从这一角度看，任务感知质量评估可以视为任务无关质量评估的进一步深化。任务无关评估给出数据本体层面的基础质量画像，而任务感知评估则根据任务目标对不同质量维度赋予不同权重，并据此判断数据是否适合当前应用。下面介绍两类具有代表性的任务感知质量评估方法，并分析如何将其应用于轨迹数据。

TFDV[12]是面向机器学习流水线的数据分析与验证工具，常用于检查训练数据、验证数据和线上服务数据之间的一致性。它通过数据统计信息和模式约束识别字段缺失、类型错误、异常取值及分布变化，并支持训练与服务偏斜和数据漂移检测。

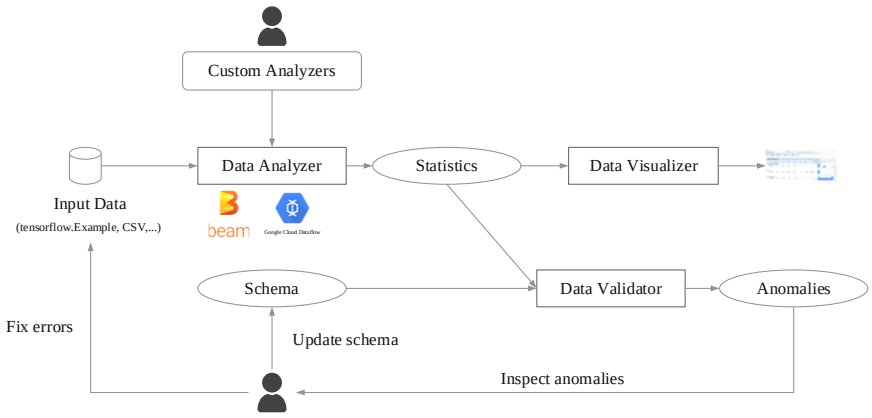


图 3.6 TFDV 的框架图（改绘自[12]）

TFDV 的框架如图 3.6 所示，其核心流程包括数据统计、模式约束、异常检测和人工反馈四个环节。数据分析器生成输入数据的字段类型、缺失比例、取值范围等统计信息，并通过数据可视化器展示。随后，系统可据此推断初始模式，供用户审核与修订。数据验证器继续将新数据统计信息与模式及参照数据比较，并输出异常报告。用户据此修复数据或调整约束，而非让模式自动接受所有异常变化。通过这一闭环机制，TFDV 能够在模型训练或线上推理前及时发现数据质量风险。

F4U[13]体现了另一类任务感知评估思想，即通过受控噪声实验评估数据质量变化对分析任务的影响。它向数据注入不同类型和程度的噪声，比较任务结果的变化，再估计质量因素与结果变化之间的关系。同一种缺陷对不同任务的影响可能不同，从而安排修复优先级。

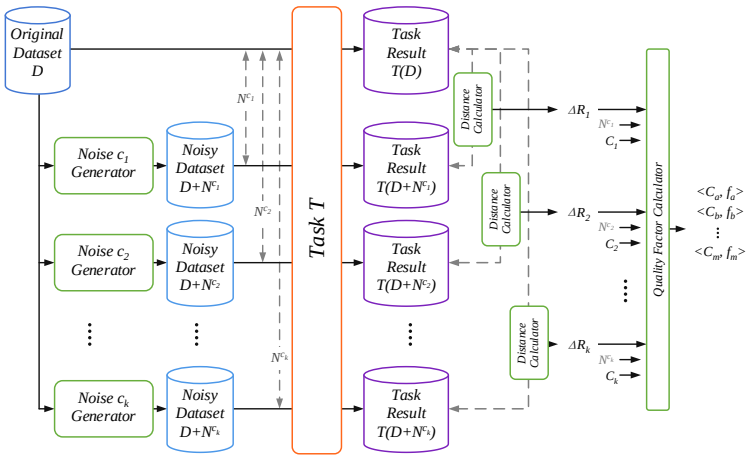


图 3.7 F4U 的框架图（改绘自[13]）

如图 3.7 所示，F4U 首先在原始数据集上执行任务，得到基准结果；随后通过不同的噪声生成器，向数据中加入缺失、异常、扰动等质量问题，形成多组带噪数据集。接着，系统将同一任务分别作用于这些带噪数据，并比较其结果与原始任务结果之间的差异。差异越大，说明该类质量问题对当前任务影响越明显。最后，F4U 通过质量因子计算器分析噪声程度与

任务结果变化之间的关系，从而判断不同质量问题的重要性。F4U 并不只关注数据本身“错了多少”，而是进一步关注这些错误会在多大程度上影响具体分析任务的结果。

任务感知质量评估可采用式（3.1）所示的加权形式：

$$Q_{task}(D) = \sum_{k=1}^{m} w_k \, q_k(D) \qquad \text{（式 3.1）}$$

其中，D表示待评估数据集，m为质量指标数量，$q_k(D)$表示第k项指标经过同向化和归一化处理后的质量得分，取值范围为[0,1]，得分越高表示质量越好；w_k表示该指标对当前任务的重要性权重，满足$w_k \geq 0$且$\sum_{k=1}^{m} w_k = 1$。因此，综合得分$Q_{task}(D)$也位于[0,1]。权重由具体任务目标确定。例如，对实时调度任务而言，可提高及时性的权重；对轨迹预测任务而言，可提高时间连续性和运动学一致性的权重。

综上，任务感知数据质量评估强调质量问题与任务目标之间的关联。它并不否定任务无关评估的基础作用，而是在其基础上进一步回答“哪些质量问题对当前任务最关键”。在实际轨迹数据治理中，通常应先通过任务无关评估识别数据的基础问题，再结合任务感知评估确定质量修复和数据筛选的优先级，从而在数据清洗成本与任务性能收益之间取得平衡。

3.2.3. 轨迹专用质量评估框架

前述方法均属于经典的数据质量评估方法，能够在一定程度上迁移到轨迹数据场景中。然而，这些方法本身并不是专门为轨迹质量评估设计的。具体而言，TsQuality 主要面向一般时间序列数据，TFDV 主要服务于机器学习流水线验证，F4U 则分析质量变化对任务结果的影响。它们虽然能够为轨迹质量评估提供有益借鉴，但并未充分考虑轨迹数据自身的特殊性，例如轨迹点之间的时空连续关系、移动对象的运动约束、道路网络等地理拓扑背景，以及轨迹数据在空间和时间上的分布偏差。因此，若直接将通用数据质量评估方法套用于轨迹数据，往往难以全面刻画轨迹数据中的典型质量问题。

为解决上述不足，Zhu 等研究者提出了面向轨迹数据的数据质量评估系统 T-Assess[3]。T-Assess 的核心目标是针对轨迹数据的时空序列特征、地理约束特征和应用需求，构建更加全面、高效和可扩展的轨迹质量评估框架。该系统认为，轨迹数据质量问题不仅表现为一般意义上的缺失值、重复值或异常值，还表现为空间位置错误、时间戳异常、轨迹断裂、轨迹偏移、道路网络约束违反以及时空分布不均衡等轨迹特有问题。

具体而言，T-Assess 将批量轨迹数据集记为$\mathcal{T}_B = T^1, T^2, \ldots, T_N$，其中$N$为轨迹数。每条轨迹$T = \langle p^1, p^2, \ldots, p_n \rangle$包含$n$个采样点，$p_i = (x_i, y_i, t_i)$分别记录经度、纬度与时间戳。后文以

$p_i.x$、$p_i.y$、$p_i.t$访问点的各分量。

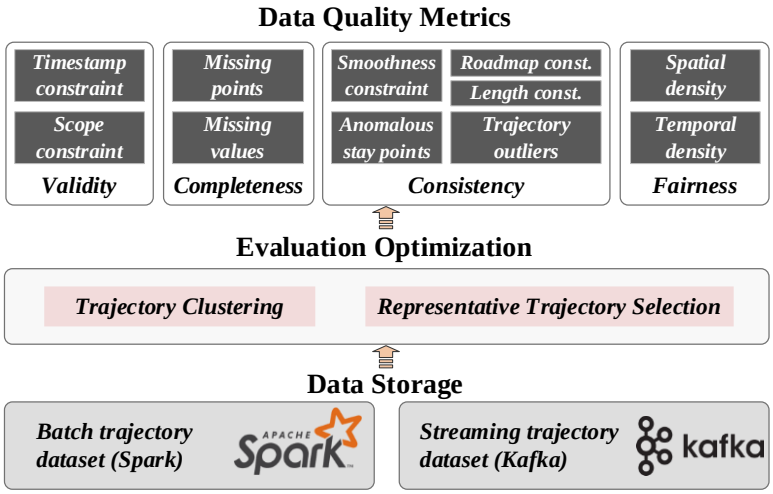


图 3.8 T-Assess 的框架图（改绘自[3]）

如图 3.8 所示，T-Assess 由数据存储、评估优化和质量指标三层组成。系统分别统计各项约束的违反点数与违反轨迹数，空间和时间覆盖则通过密度指标报告。

在数据存储与处理层，对于历史轨迹数据，系统采用 Spark 处理批量轨迹数据集；对于实时产生的轨迹数据，系统采用 Kafka 支持流式轨迹数据接入。离线评估对轨迹数据集$\mathcal{T}_B$执行约束检查，适用于历史轨迹数据质量分析、轨迹清洗前诊断和城市交通数据治理等场景；在线评估以$\mathcal{T}_S$表示持续到达的轨迹数据集，给定窗口长度W和当前时刻t_c，仅对$[t_c - W, t_c]$内的点及相应轨迹统计违反数，适用于实时交通监测和轨迹流质量监控等场景。

中间层为评估优化层，其核心思想是利用轨迹之间的空间相似性来进行网格聚类，从而减少冗余计算。T-Assess 首先将空间划分为多个网格，以网格 ID 替换点坐标。对于当前轨迹T，若它与某个已有簇的中心轨迹共同经过至少O个网格，则将其加入该簇；否则建立新簇，并将当前轨迹作为簇中心。随后，对簇计算归一化规模，选择代表性轨迹进行约束检查，再用这些结果近似估计同簇其他轨迹的质量，从而减少需要逐条检查的轨迹数量。

最上层为质量指标体系层。T-Assess 构建了面向轨迹数据的质量指标体系，主要包括有效性、完整性、一致性和公平性四个维度。

1. 有效性关注轨迹记录是否满足基本的时间和空间约束。T-Assess 从时间戳顺序和坐标范围两个方面进行检查：时间戳应随记录顺序保持非递减，即$p_{(i+1)}.t \geq p_i.t$，以识别时间乱序的情况；经纬度$p_i.x$、$p_i.y$则应落在预先指定的区域边界内，以识别越界坐标。

2. 完整性关注移动过程是否被充分记录，既包括轨迹点是否缺失，也包括已有点的坐标字段是否齐全。T-Assess 根据相邻记录的时间间隔检查缺点：对于固定采样间隔Δt，若

$p_{(i+1)}.t - p_i.t \geq 2\Delta t$，则将该区间标记为可能缺点。例如，正常每 10 秒记录一次，而两次记录相隔 20 秒或更长，就需要核查中间是否漏采。此外，系统逐条检查经度、纬度是否为空，分别统计缺点区间和字段缺失记录。

3. 一致性是 T-Assess 中最能体现轨迹数据特殊性的维度。它关注轨迹点之间、轨迹与轨迹之间，以及轨迹与地理环境之间是否符合合理的运动规律和语义约束。具体而言，一致性指标包括平滑性约束、轨迹长度约束、轨迹离群检测、异常停留点检测等。例如，轨迹离群检测先在其他轨迹中，为每个点查找距离小于σ的邻点；邻点数少于η时，将该点标记为离群点；一条轨迹中的离群点数超过ρ时，再将整条轨迹标记为离群候选。对于轨迹内部，系统检查位置是否突变、累计长度是否达到预设要求；对于地理环境，系统结合停留点聚类与地图匹配，检查停留位置及轨迹点与道路段的对应关系。

4. 公平性从数据分布偏差的角度评估轨迹数据质量，T-Assess 采用空间相对密度和时间相对密度衡量这种差异。以空间维度为例，先计算某一区域每单位面积内的轨迹点数，再除以全局平均值，即：

$$D_s(Q_s) = \frac{|\mathcal{T}_B(Q_s)|/A(Q_s)}{M/A} \qquad \text{（式 3.2）}$$

其中，Q_s为待评估的空间区域，$D_s(Q_s)$为该区域的空间相对密度，$|\mathcal{T}_B(Q_s)|$和M分别为区域内和数据集的点数，$A(Q_s)$和A分别为区域面积和总面积。时间相对密度采用同样的计算方式，将面积替换为时长。相对密度低于 1 则表示局部覆盖低于平均水平。

T-Assess 代表了面向轨迹特征的专用质量评估框架。它表明，轨迹数据质量评估不能只停留在一般意义上的缺失值、重复值和异常值检测，而应充分考虑轨迹数据的时空连续性、运动规律、地理拓扑依赖、实时处理需求和时空分布偏差。只有将这些因素纳入统一评估体系，才能更加全面地判断轨迹数据是否真实、完整、合理，并是否适用于后续分析任务。

综上，轨迹质量评估的发展呈现出明显的层次化趋势。早期方法主要关注单点错误和简单规则违反，如坐标越界、时间乱序和字段缺失；随后，通用数据质量框架开始从完整性、一致性、及时性、有效性以及任务效用等角度提供更加系统的评估思想；进一步地，以 T-Assess 为代表的轨迹专用方法将质量评估与轨迹数据自身的时空属性、道路网络约束和分布覆盖问题结合起来，形成更加面向轨迹场景的综合质量评估体系。

3.3 轨迹清洗

轨迹观测中的定位偏差、时间记录错误和重复记录，可能使计算得到的位移、速度及方

向变化偏离实际运动过程。清洗时既要处理错误观测，也要保留急转弯、短暂停留等真实变化。因此，需要结合采集条件、运动规律与地理环境判断可疑记录并选择相应的处理方式。

轨迹清洗方法主要包括传统方法和基于深度学习的方法。传统方法利用规则约束、平滑滤波和统计模型识别与修正异常，也可组合多种机制处理复合噪声；深度学习方法则通过预测、重构或生成概率学习数据中的时序关系，为异常识别和修复提供依据。本节首先界定轨迹清洗的输入输出与基本要求，随后介绍两类方法的代表算法及其处理过程。

3.3.1. 轨迹清洗概述

轨迹清洗针对已有观测中的错误、噪声和不一致记录，进行识别、修正或筛选。设输入轨迹为 $T = \langle p_1, \dots, p_n \rangle$，其中 $p_i = (s_i, t_i)$，$s_i = (x_i, y_i)$ 为空间位置，t_i 为时间戳。清洗输出为轨迹 T^*，并可附带异常标记和修改记录。根据处理方式，T^* 可以保持原有点数，也可以通过删除无效记录形成原始轨迹的子序列。

对于点级异常检测，记 u_i 为观测点 p_i 的异常得分，数值越大表示越可疑。给定阈值 η，可将异常候选集合表示为：

$$A = \{ i \mid u_i > \eta,\ 1 \le i \le n \}$$

异常得分可以由规则违反程度、预测残差、重构误差或生成概率转换得到，并统一为数值越大表示越可疑。例如，可采用负对数生成概率构造异常得分。集合 A 表示需要进一步核查的位置，并不意味着其中所有观测都应被删除。例如，真实绕行或急转弯也可能偏离常见运动模式，需要结合采集条件与外部证据判断。对于连续异常，还可将检测对象扩展为轨迹片段。

清洗约束主要包括字段与时间约束、运动约束，以及适用的地理约束。以下以时间戳已校验、且保持点数不变的位置修复为例，输出为 $T^* = \langle (s_1^*, t_1), \dots, (s_n^*, t_n) \rangle$。若 $t_i > t_{i-1}$，相邻修复点之间的平均速度为：

$$v_i^* = \frac{d(s_{i-1}^*, s_i^*)}{t_i - t_{i-1}}, \qquad 2 \le i \le n$$

其中，$d(\cdot,\cdot)$ 为与坐标系统相适应的距离函数。速度约束可写为 $0 \le v_i^* \le v_{max}$，其中 v_{max} 根据移动对象和应用条件设定。加速度、方向变化及道路通行条件也可形成相应约束，共同限定修复结果的可行范围。经纬度坐标不能直接按米制平面坐标计算距离，应采用地表距离或先进行适当投影。

记满足给定约束的位置序列集合为 Ω。对于约束驱动的位置修复，可在可行范围内尽量减小对原始观测的改动：

$$S^* = \arg\min_{S' \in \Omega} \sum_{i=1}^{n} w_i \; d(s_i', s_i)^2$$

其中，$S' = \langle s_1', \ldots, s_n' \rangle$ 为候选修复序列，S^* 为选定结果，$w_i > 0$ 表示原始观测的可信权重。较大的权重意味着修改该点需要付出更高代价，有助于保留可信观测；较低可信度的点则允许作较大调整。该表述体现了满足约束与控制修改幅度之间的权衡，具体方法也可采用滤波值、预测值或重构值生成修复候选，再进行约束检查。

采用删除策略时，输出可表示为 $T^* = \langle p_i \mid i \in I \rangle$，其中 I 为按原始顺序排列的保留索引集合；采用数值修复时，则需记录被修改的位置及替换值。若证据不足，也可保留原始记录并附加可疑标记。清洗首先判断已有观测是否需要改动，轨迹恢复则侧重推断未观测的位置或路径，两者可以在处理流程中结合。

评价清洗结果时，应同时检查错误处理效果与可信信息保留程度。有参考数据时，可评价异常识别与位置修复误差；缺少真实参照时，可报告约束违反情况、修改比例及抽样核查结果。修改后的轨迹更加平滑或违反约束的数量减少，并不足以单独证明清洗准确，还需检查是否损失真实的转向、停留和速度变化。

3.3.2. 传统轨迹清洗

传统轨迹清洗方法主要围绕“异常检测—异常修正—轨迹连续性保持”这一思路展开。由于轨迹数据同时具有一般数据问题和时空序列特征，相关方法通常从规则约束、信号平滑、几何统计等角度进行处理。其中，一般数据问题包括缺失、重复和噪声等；时空序列特征则体现在时间连续性、空间邻近性和运动约束等方面。

不同方法的侧重点有所差异。启发式方法强调基于经验规则快速识别明显异常；平滑滤波方法侧重削弱随机噪声；几何统计方法关注轨迹局部形态中的离群特征；混合式方法融合不同技术的优势以应对复杂的复合型噪声。下面对几类典型方法进行介绍。

（1）启发式清洗方法

启发式清洗方法是传统轨迹清洗中最直观、应用也较为广泛的一类方法。该类方法通常根据移动对象的物理运动规律或领域经验设定规则阈值，例如最大速度[14]、最大加速度[15]、最大空间跳跃距离和最小采样间隔等。当相邻轨迹点之间的速度或位移超过合理范围时，系统将相关点标记为待核查的异常候选，并进一步进行删除、修正或分段处理。

以 SCREEN 算法[16]为代表，相关研究将速度约束引入轨迹清洗过程，通过判断相邻点之间是否满足预设速度范围来识别异常记录。这类方法的优势在于规则明确、实现简单、计

算效率较高，适合用于大规模轨迹数据的初步筛查。但其局限也较为明显：清洗效果高度依赖阈值设定。阈值过宽可能漏检小幅异常，阈值过严则可能误删真实但特殊的移动行为。

算法 3.1 展示了流式场景下的 SCREEN 算法，该算法的核心思想是利用速度约束对当前点进行局部修正，使清洗后的相邻点之间满足合理的速度范围。首个点没有前序已清洗点，保持原值作为起点（第 1 行），此后从第 2 个点起逐点处理。具体而言，对于当前点x_k，先以前一个清洗完成的点x_{k-1}^*为基准，计算其允许取值范围$[low, high]$（第 4-5 行），其中s_{min}和s_{max}分别表示速度（一维数值序列 x_k 的有符号变化率，可取负值）的下界和上界；该标量形式不能直接作为二维位置的欧氏速度约束。然后，在滑动窗口w内，根据速度约束反推出当前点x_k的一个可能取值，并将这些可能取值加入候选集合C（C 保留重复值，按多重集处理；第 6-9 行）。为了降低单个噪声点对修复结果的影响，算法取候选集合的中位数mid作为当前点的参考修复值（第 10 行）。若mid超出允许范围$[low, high]$，则将其修正到边界（第 11-14 行）；否则直接采用该值（第 16 行）。通过这种方式，SCREEN 算法既利用前一个已清洗点保证局部速度约束，又借助滑动窗口中的后续观测增强修复结果的稳定性，从而在满足运动合理性的同时尽量减少对原始数据形态的过度修改。

算法 3.1 流式场景下的 SCREEN 算法

输入： 待清洗数据集x，时间戳序列t，速度阈值s_{max}, s_{min}，滑动窗口w
输出： 清洗完成的数据集x^*

1: $n \leftarrow length(x)$，$x_1^* \leftarrow x_1$ // 首点没有前序已清洗点，保持原值
2: **for** $k \leftarrow 2$ **to** n **do**
3: $C \leftarrow \{x_k\}$ // 初始化候选集合，保留当前原始值
4: $low \leftarrow x_{k-1}^* + s_{min} \times (t_k - t_{k-1})$ // 根据前一个已清洗点计算当前点的下界
5: $high \leftarrow x_{k-1}^* + s_{max} \times (t_k - t_{k-1})$ // 根据前一个已清洗点计算当前点的上界
6: **for** $i \leftarrow k+1$ **to** n **do**
7: **if** $t_i > t_k + w$ **then**
8: **break**
9: $C \leftarrow C \cup \{x_i + s_{max} \times (t_k - t_i)\} \cup \{x_i + s_{min} \times (t_k - t_i)\}$ // 生成候选修复值
10: mid ← median(C) // 取候选集合的中位数作为参考修复值
11: **if** $mid < low$ **then**
12: $x_k^* \leftarrow low$ // 若低于允许范围，则修正为下界
13: **else if** $mid > high$ **then**
14: $x_k^* \leftarrow high$ // 若高于允许范围，则修正为上界
15: **else**
16: $x_k^* \leftarrow mid$
17: **return** x^* // 由各点的修复值依次组成

（2）平滑滤波清洗方法

此外，常见的轨迹数据清洗方法还有平滑滤波算法和自回归模型。平滑滤波主要利用历史观测或邻近点估计当前状态，以削弱随机测量误差。常见方法包括移动平均[17]、指数加权移动平均[18]和卡尔曼滤波[19]。而自回归模型可通过预测残差进行异常筛查[20]，当异常点被确认后，可利用预测值形成修复候选。下面以典型的卡尔曼滤波算法为例进行介绍。

卡尔曼滤波将位置、速度等运动状态视为隐变量，将设备定位结果视为含噪观测，通过预测与更新递推估计状态。在线性状态模型和给定噪声协方差条件下，它综合运动模型与观测的不确定性，适合处理连续运动中的随机误差。

具体而言，设移动对象在时刻t的真实状态为$\mathrm{x_t}$，观测到的轨迹点为$\mathrm{z_t}$。在线性模型下，卡尔曼滤波首先根据上一时刻的状态估计进行预测，得到当前状态的先验估计，如式（3.3）所示：

$$\hat{x}_{t|t-1} = F\hat{x}_{t-1|t-1} \quad \text{（式 3.3）}$$

其中，F为状态转移矩阵，$\hat{x}_{t|t-1}$为当前状态的先验估计。随后，算法利用当前观测值 z_t，按照式（3.4）对先验估计进行更新，得到当前状态的后验估计：

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t(z_t - H\hat{x}_{t|t-1}) \quad \text{（式 3.4）}$$

其中，H为观测矩阵，K_t为卡尔曼增益，由预测误差协方差与观测噪声协方差共同确定。式（3.3）和式（3.4）展示了状态估计的预测与更新过程，卡尔曼增益的计算以及误差协方差的预测和更新递推详见文献[19]。标准卡尔曼滤波不会自动将大残差识别为异常并降低其权重，在异常清洗时，可另设残差门限或采用鲁棒更新。

然而，平滑滤波方法高度依赖于运动模型和噪声假设。当移动对象发生急转弯、突然停车或快速变速时，如果模型仍假设其保持平稳运动，可能会将真实行为误认为噪声并进行过度平滑。因此，在实际应用中，平滑滤波算法通常适合用于随机定位误差较明显、运动过程相对连续的轨迹清洗场景。

（3）几何统计清洗方法

几何统计清洗方法[21,22]主要从轨迹点之间的空间结构和统计分布特征出发识别异常。轨迹可以看作由一系列时空点组成的线状结构，相邻点距离、方向角、转向角、局部曲率、速度变化率和邻域密度等指标均可用于判断轨迹是否合理。例如，若某一轨迹点与前后点形成明显尖峰，且该点明显偏离局部主运动方向，则可能是定位漂移造成的孤立异常点。

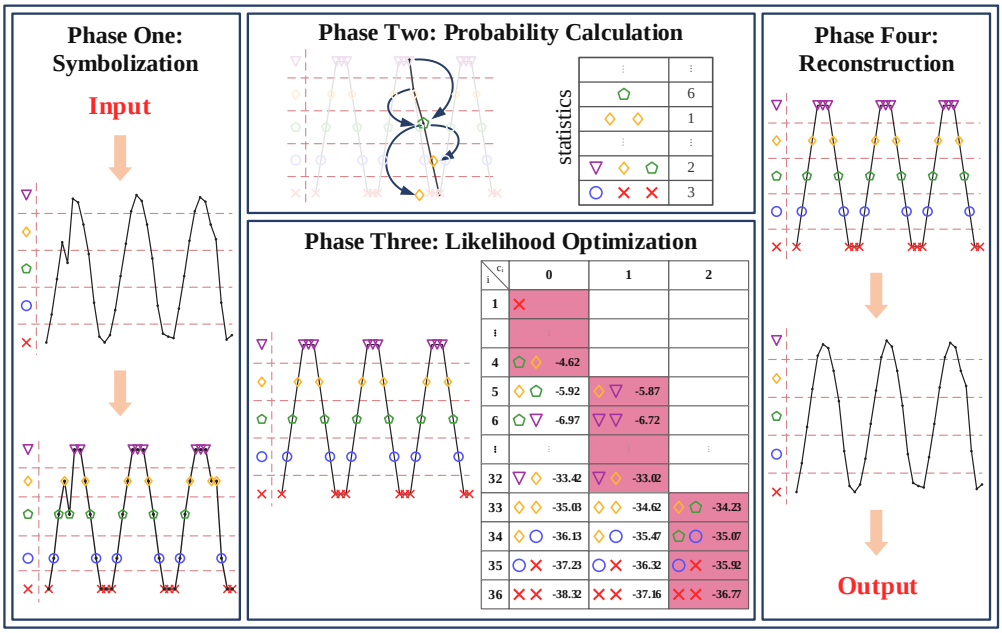


图 3.9 Akane 算法的框架图（改绘自[21]）

以 Akane[21]算法为例，该算法不是简单判断某个点是否超过某个阈值，而是从序列中挖掘“反复出现的局部模式”。如图 3.9 所示，它将数值序列离散化为符号序列，用马尔可夫模型刻画重复的局部模式，并在给定修改预算内寻找似然更高、困惑度更低的修正序列。因此，清洗过程可以理解为在尽量少修改原始数据的前提下，寻找一个困惑度更低、统计上更符合局部模式的序列。

具体而言，若将轨迹点的位置、速度、方向角、转向角或局部曲率等指标看作时间序列的特征列，Akane 的思想则可以用于识别那些与前后运动模式不协调的异常点。例如，在一段平稳前进的轨迹中，某个点突然偏离主运动方向，随后轨迹又回到原有趋势，这种局部结构在统计模式上较为罕见，会导致对应符号片段的概率下降。Akane 通过马尔可夫统计建模估计局部符号模式的出现概率，并在给定清洗预算下最大化修正后序列的似然，从而决定哪些点应被修改，而不是对所有可疑点机械地套用固定阈值。

与传统的启发式阈值算法相比，几何统计算法更强调上下文一致性和数据自身的统计规律。它不仅关注某个点是否“看起来异常”，还关注该点所在的局部片段是否符合历史中反复出现的运动模式。因此，这类方法可用于分析空间漂移、孤立跳点和连续异常片段。不过，当轨迹缺乏明显重复模式、采样较稀疏，统计模式可能不足以区分真实行为与异常噪声，仍需要结合速度约束、地图约束或业务规则共同判断。

（4）混合式清洗方法

混合式清洗方法是在单一清洗方法基础上发展起来的一类综合性方法[23,24,25]。该类方法

结合多种清洗机制，在同一框架下利用规则约束、统计分布和多变量关系处理异常。与单一方法相比，混合式方法通常不再只判断某个轨迹点是否超过固定阈值，而是同时考虑该点是否满足物理约束、是否符合局部或全局统计规律、是否破坏轨迹的连续形态，以及是否与其他相关变量保持一致。这样既可以利用硬性规则排除明显不合理的数据，又可以借助统计模型发现较隐蔽的小幅异常，从而提升清洗结果的稳定性。

SHoTClean[25]是混合式清洗研究中代表性的算法，其基本思想可以概括为“以硬约束限定可行空间，以软约束选择最优路径”。其中，硬约束（如速度、加速度）用于强制过滤不合理的物理连接，软约束（如偏离均值或局部模式的程度）则量化观测点的统计置信度。两者分别表达“是否允许连接”和“在可行连接中更偏好哪些观测”。

与逐点判断异常不同，SHoTClean 将轨迹清洗转化为有向无环图（Directed Acyclic Graph，DAG）中的最大权重路径搜索问题。在该 DAG 中，节点代表原始轨迹点；若两节点在时间序列上相邻或跨度较小，且其派生的时空变量（如速度）满足硬约束，则建立有向边。边权重由软约束得分与时间衰减因子共同决定。最终，图中权重最大的路径所包含的节点被视为“正常轨迹序列”，其余节点即被判定为异常脏点，并使用线性插值方法对脏点进行清洗修复。

如算法 3.2 所示，SHoTClean 的离线最大权重路径搜索流程如下：首先，算法为每个轨迹点计算软约束得分（第 1 行；μ、σ分别为各维均值与标准差，σ为 0 时取一个极小正数以避免除零）；接着，利用速度上下界$[V_{\min}, V_{\max}]$对点对之间的转移合理性进行硬约束校验，筛选出各节点的合法前驱节点集合（第 2-5 行）。为降低图搜索的计算复杂度，算法采用动态规划求解最大权重路径：依次遍历前驱节点，在状态转移时融入时间衰减因子γ，计算候选路径权重并更新最大累计得分$dp[i]$及最优前驱指针$prev[i]$（第 6-11 行）；遍历过程中用 best_score 与 e_id 记录全局最大得分及其终止节点（第 6 行初始化，第 12-13 行更新）。最后，算法沿$prev$指针回溯得到最大权重路径（第 14 行），不在该路径上的点判定为脏点（第 15 行）。该动态规划避免了枚举全部候选路径，但需对每个点检查其全部前驱，时间复杂度为$O(n^2)$；面对长周期、大规模轨迹，可结合前驱范围限制或分段处理控制开销。

算法 3.2 SHoTClean 算法中离线场景下的最大权重路径搜索

输入： 待清洗数据集X，时间戳序列t，速度阈值V_{max}, V_{min}，偏离系数α，衰减因子γ
输出： 待清洗的脏点dirty point

$s[i] \leftarrow \exp\left(-\alpha \cdot \left\|\frac{x_i-\mu}{\sigma}\right\|_2\right)$ // 通过均值和标准差计算每个点的软约束得分
for $i \leftarrow 1$ **to** n **do**
  for $j \leftarrow i-1$ **to** 1 **do**
    if $\frac{\|x_i-x_j\|_2}{t_i-t_j} \in [V_{min}, V_{max}]$ **then** // 判断两点之间是否满足速度硬约束
      $preds[i]$.append(j) // 将节点j加入到节点i的前驱节点中
$best_score \leftarrow -\infty, e_id \leftarrow 0$；**for** $i \leftarrow 1$ **to** n **do**
  $dp[i] \leftarrow s[i],\ prev[i] \leftarrow i$ // 初始化路径得分和前驱节点
  foreach $j \in preds[i]$ **do** // 遍历当前节点i的所有合法前驱节点j
    $cand \leftarrow dp[j] + s[i] \cdot \gamma(i-j)$ // 计算从节点j转移到节点i的候选路径权重
    if $cand > dp[i]$ **then**
      $dp[i] \leftarrow cand, prev[i] \leftarrow j$ // 更新节点i的最大路权，并记录前驱节点j
  if $dp[i] > best_$score **then**
    $best_$score $\leftarrow$ dp$[i]$, $e_$id $\leftarrow$ i // 更新最大路权，并记录终止节点
$Route \leftarrow$ backtrack_path$(prev, e_{id})$ // 通过反向回溯，得到最大权重路径$Route$
return dirty point $\{X_1, X_2, \dots, X_n\} \setminus Route$ // 将不在$Route$中的点判定为脏点并返回

SHoTClean 的优势在于能够同时处理明显异常和隐蔽异常。对于大幅跳变、速度超限等问题，硬约束可以直接排除不合理连接；对于没有明显违反物理规则、但偏离正常轨迹模式的小幅漂移点，软约束得分也可以帮助算法识别其异常性。同时，该方法遵循最小修改原则，不会像平滑滤波方法那样对整条轨迹进行整体调整，而是尽量保留可信观测点，仅对被判定为异常的点进行处理，因此能够较好地保持原始轨迹的整体形态。

混合式清洗算法是轨迹数据清洗的重要发展方向。它通过融合物理约束、统计规律、时间连续性和多变量关系，可弥补单一方法的部分局限。以 SHoTClean 为代表的方法将硬约束与软约束统一到路径优化框架中，有助于分析明显跳变及部分较小偏移。未来，随着数据规模扩大和应用场景复杂化，混合式清洗算法仍需在参数自适应、实时处理效率和多源语义信息融合等方面进一步优化。

3.3.3. 基于深度学习的轨迹清洗

随着轨迹数据规模的持续扩大以及移动场景复杂性的不断提升，传统依赖人工阈值、统计假设或单一几何规则的轨迹清洗方法逐渐显现出一定局限。一方面，真实轨迹数据中的异常并不总是表现为明显的速度突变或空间跳跃，部分异常可能仅体现为局部轻微偏移、短时漂移，或多维特征之间的不一致；另一方面，轨迹数据通常具有复杂的时序依赖、空间关联

和行为语义，仅依靠固定规则难以充分刻画其潜在变化规律。

因此，近年来深度学习方法逐渐被引入轨迹清洗任务。该类方法能够通过神经网络自动学习轨迹序列中的时空特征、上下文关系和潜在分布模式，从而实现异常点识别、缺失片段补全以及噪声轨迹修正。其中，部分异常检测方法本身会产生预测值、重构值或插补值。对已核验的异常位置，可以采用这些估计值替换原始观测，并保留正常位置，形成“检测—候选修复—约束校验”的清洗流程。

（1）基于生成概率的清洗算法

GM-VSAE[26]是一种面向在线异常轨迹检测的方法。其基本思想是学习正常轨迹的路线模式，并根据待检测轨迹在这些模式下的生成可能性识别异常。由于同一起讫区域之间可能存在多条合理路线，模型采用高斯混合先验表示不同的正常路线类型，避免用单一模式描述所有轨迹。在训练阶段，循环神经网络编码轨迹的序列信息，变分自编码器将其映射到连续潜在空间，并通过解码器重构轨迹，使模型学习路线的整体特征及轨迹点之间的序列依赖。

在检测阶段，模型利用解码器评估观测轨迹在所学路线模式下的生成概率。解码器根据潜在路线表示和已有序列信息，为后续位置赋予概率，并将各步的概率信息汇总为对已观测序列的评价。当轨迹与某种正常路线模式较为一致时，其生成可能性通常较高；当轨迹逐渐偏离所学模式时，其生成可能性降低，异常程度随之提高。随着新的轨迹点到达，模型持续更新对当前轨迹前缀的评价，因此无需等待整条轨迹结束即可进行在线异常检测。

在轨迹清洗中，GM-VSAE 可用于筛选偏离正常路线模式的轨迹，再结合速度变化、路网可达性等检查定位可能的观测错误，并进行相应修复。

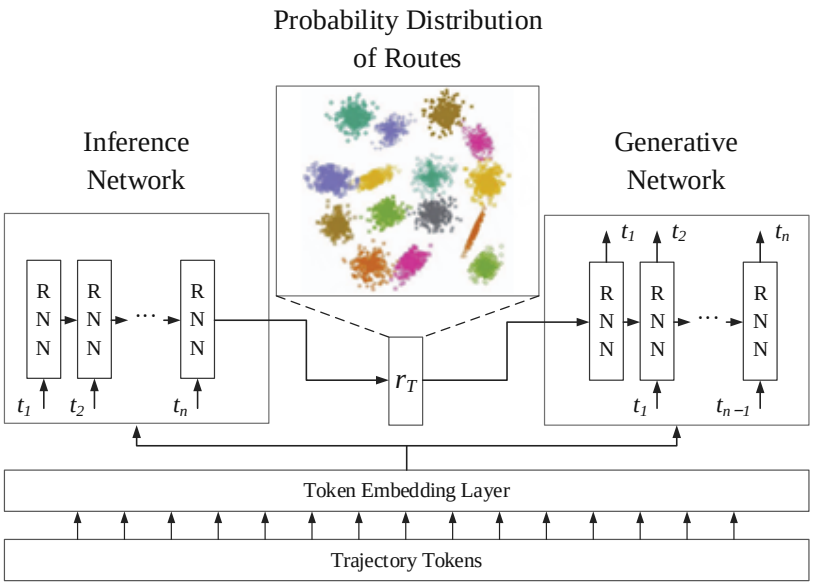


图 3.10 GM-VSAE 算法的框架图（改绘自[26]）

然而，基于循环神经网络的清洗方法也存在一定不足。当轨迹序列较长时，模型可能难以充分捕捉远距离依赖关系；同时，模型训练通常依赖较大规模的高质量样本。如果训练数据本身包含较多噪声，模型可能学习到错误的运动模式，进而影响轨迹清洗结果的可靠性。

（2）基于重构误差的清洗算法

基于 Transformer 的轨迹清洗算法通常包括数据表示、序列建模、异常检测和轨迹恢复四个基本步骤。TranAD[27]是面向多变量时间序列的 Transformer 异常检测模型，其框架如图 3.11 所示。模型结合历史上下文与当前窗口进行重构，并利用重构误差计算异常得分。将其用于轨迹清洗前，需要选择特征、处理不规则采样，并用验证数据确定异常阈值。

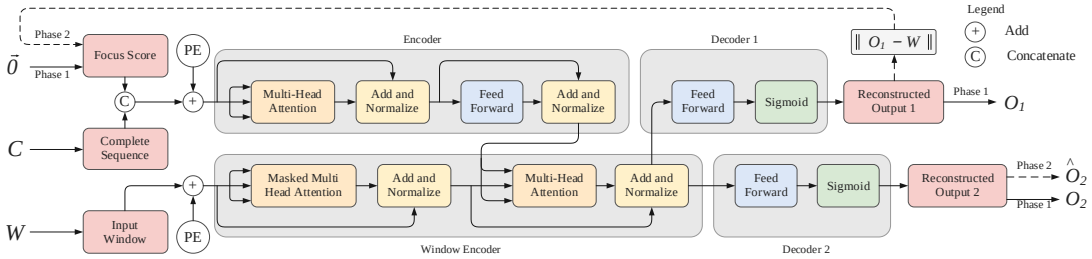


图 3.11 TranAD 的框架图（改绘自[27]）

TranAD 采用两阶段重构，以第一阶段的误差形成焦点信息，再将该信息用于第二阶段的重构，并结合对抗训练增强异常区分。清洗时，可把超过阈值且通过运动约束核验的位置标为待修复点，用对应的重构值作为替代候选。若重构对象是速度等派生特征，还需把修复结果与位置序列联合校准，保持各变量一致。

基于 Transformer 的方法能够较好地捕捉长距离时序依赖和复杂变量关系，适用于长轨迹、多维轨迹以及具有明显上下文关联的轨迹数据。但其不足也较为明显：模型训练成本较高，对数据规模和计算资源要求较大；同时，由于深度模型内部机制相对复杂，其清洗结果的可解释性通常弱于基于规则或统计假设的方法。

（3）基于扩散插补的清洗算法

生成式模型主要通过学习轨迹数据的潜在分布，对缺失、异常或受噪声污染的轨迹片段进行生成和修复。常见结构包括生成对抗网络（Generative Adversarial Network，GAN）、变分自编码器（Variational Autoencoder，VAE）和扩散模型（Diffusion Model）。其基本思路是利用模型学习正常轨迹的分布特征，当输入轨迹存在噪声或缺失时，模型可以根据上下文生成更符合真实分布的轨迹点或轨迹片段。

ImDiffusion[28]是一种基于扩散插补的多变量时间序列异常检测模型。如图 3.12 所示，该方法通过掩码遮蔽序列中的部分观测值，并利用扩散模型进行插补。前向扩散过程逐步向

数据加入高斯噪声，模型通过训练学习噪声预测，并在反向过程中逐步去噪，估计被遮蔽位置的取值。随后，模型利用插补值与实际观测值之间的误差识别异常，并综合多个去噪步骤的检测结果，提高检测的稳健性。将这一思路用于轨迹清洗时，可先利用插补误差筛查可疑观测，再结合路网可达性、速度范围等条件判断是否需要修复。插补值可作为候选修复值，经校验后用于替换错误观测。

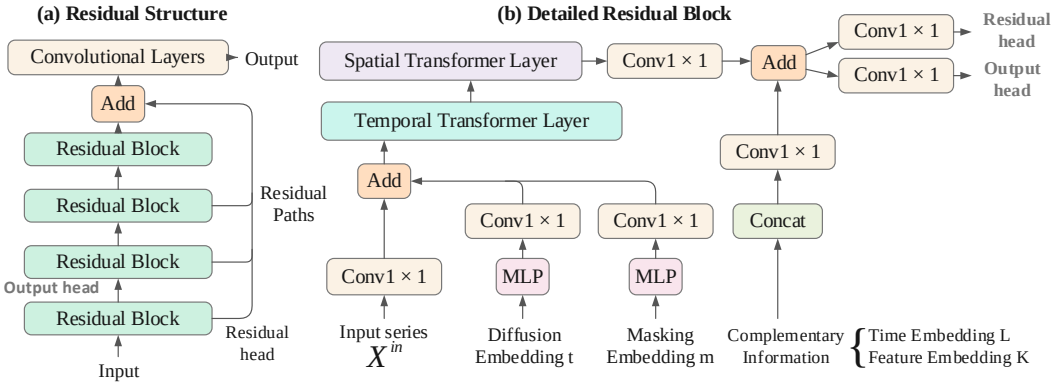


图 3.12 ImDiffusion 的框架图（改绘自[28]）

训练数据中的噪声可能影响模型的插补结果。将这些结果用于轨迹清洗时，应结合前后观测、运动约束和适用的道路条件进行核验，再决定是否替换原始记录。

总体来看，深度学习方法为轨迹清洗提供了更强的非线性建模能力和自动特征学习能力。与传统方法相比，深度学习方法能够从大规模数据中学习复杂的时序模式、空间关联和多变量依赖关系，尤其适用于噪声类型复杂、轨迹规模庞大和移动行为多样化的应用场景。

然而，深度学习方法并不能完全替代传统清洗方法。一方面，其训练过程通常需要大量数据和较高计算成本；另一方面，模型结果的可解释性较弱，且容易受到训练数据质量的影响。如果训练数据中本身包含大量异常，模型可能将异常模式误认为正常规律。因此，在实际轨迹清洗中，更合理的做法是将深度学习方法与传统约束方法结合使用：利用速度、加速度、时间连续性和路网规则等硬约束保证清洗结果的基本合理性，同时利用深度模型学习轨迹数据中的复杂软约束和潜在模式。通过这种方式，可以在清洗准确性、模型鲁棒性和结果可解释性之间取得更好的平衡。

3.4 轨迹压缩

轨迹数据的持续采集会产生大量位置记录，其存储、传输及查询成本随数据规模增长而增加。在较高采样频率下，部分相邻观测对轨迹表示提供的信息较为接近，具有缩减空间；但不同采样点对运动过程的表达作用有所差异，统一降低采样频率可能遗漏转向、停留或速

度变化等局部信息。因此，需要根据轨迹特征确定压缩方式并评价数据缩减造成的信息损失。

现有轨迹压缩研究主要包括传统方法与深度学习方法。传统方法依据预设误差度量，通过关键点选择或分段近似生成压缩结果；深度学习方法则利用训练数据学习紧凑表示或点选择策略。不同方法在输出形式、误差控制和计算开销方面有所差异。本节首先介绍轨迹压缩的问题定义与评价依据，随后分别讨论两类方法的代表算法及其压缩过程。

3.4.1. 轨迹压缩概述

轨迹压缩旨在以较小的数据表示近似原始轨迹，并控制由此产生的信息损失。设原始轨迹为 $T = \langle p_1, \dots, p_n \rangle$，其中 $p_i = (x_i, y_i, t_i)$ 记录空间位置与时间戳。常见压缩结果包括由部分原始采样点构成的简化轨迹，以及由模型生成的紧凑潜在表示。两者均减少表示规模，但输出形式和信息恢复方式有所不同。

以保留原始采样点的轨迹简化为例，输出为 $T' = \langle p_{s_1}, \dots, p_{s_m} \rangle$，其中 $1 = s_1 < \cdots < s_m = n$，即保留首尾点及采样点的原始顺序。简化轨迹通常利用相邻保留点之间的连线近似原始运动过程，保留率为 m/n。保留点越少，表示规模越小，但未保留部分的运动细节也可能损失。因此，关键点选择需要同时考虑表示规模与近似误差。

根据约束条件，轨迹简化可表述为两类问题。第一类给定保留点预算 W，在点数不超过预算的条件下，寻找误差最小的简化轨迹：

$$\min_{T'} D(T, T') \qquad |T'| \le W$$

其中，$D(T, T')$ 表示简化轨迹与原始轨迹之间的差异，$|T'|$ 为保留点数。第二类给定允许误差 (ε)，在满足误差要求的条件下，尽量减少保留点数：

$$\min_{T'} |T'| \qquad D(T, T') \le \varepsilon$$

上述两类问题均要求输出满足首尾点保留和顺序约束，分别对应存储预算受限与误差容限明确的需求。不同算法可能采用精确搜索、递归分裂或局部贪心等策略，其优化结果和计算开销也有所差异。

误差度量决定了压缩过程优先保持哪些信息。垂直欧氏距离（Perpendicular Euclidean Distance，PED）衡量原始点相对于近似连线的空间偏差；同步欧氏距离（Synchronized Euclidean Distance，SED）比较原始位置与同一时刻的插值位置，可反映部分与速度变化有关的偏差；方向误差则衡量近似前后的行进方向差异。因此，相同点数预算下，不同误差目标可能产生不同的关键点集合，某一指标下的较优结果也未必同时保持停留、速度变化或其

他语义信息。

对于潜在表示压缩，编码器将轨迹映射为低维向量 z，再由解码器生成重构轨迹 $\hat{T}$。此时，压缩程度主要与潜在表示的维度、数值精度及必要的辅助信息有关，信息损失则可通过原始轨迹与重构轨迹之间的差异评价。此类结果需要配合解码器恢复轨迹，其表示维度不能直接与轨迹简化的保留点数等同。

3.4.2. 传统轨迹压缩

代表性的传统轨迹压缩算法大体可分为三类：基于递归分裂的方法、基于逐步合并或点删除的方法，以及基于优化搜索的方法。前两类通常采用启发式或贪心策略，在效率和压缩质量之间进行折中；第三类则从全局优化角度寻找误差较小的简化轨迹，压缩质量较高但计算开销较大。

（1）基于递归分裂的方法

基于递归分裂的方法采用自顶向下的策略，根据近似误差逐步划分轨迹，代表算法为 Douglas–Peucker（DP）算法[29]，这类研究将垂直距离误差引入压缩过程，通过判断中间点到起终点连线的最大偏差来选择关键点。

DP 算法以原始轨迹 $\mathrm{T} = \langle \mathrm{p}_1, \mathrm{p}_2, \ldots, \mathrm{p}_\mathrm{n} \rangle$ 和距离容差 ε 为输入，输出由原始采样点组成的简化轨迹。对于当前处理的轨迹片段 $T[i{:}j]$，首先连接首尾点 p_i 和 p_j，以线段 $\overline{p_i p_j}$ 近似该片段；随后计算各中间点相对于该线段的垂直距离误差，并找出最大误差及其对应点：$k^* = \arg\max_{i<k<j} d_{\mathrm{PED}}\left(p_k, \overline{p_i p_j}\right)$，$d_{max} = d_{\mathrm{PED}}\left(p_{k^*}, \overline{p_i p_j}\right)$. 若 $d_{max} \le \varepsilon$ 片段满足容差要求，仅保留首尾点；否则，保留最大偏差点 p_{k^*}，将片段划分为 $T[i{:}k^*]$ 和 $T[k^*{:}j]$，分别递归处理。每次递归均以当前子片段的首尾连线重新计算误差，直至子片段满足容差要求，或仅包含两个相邻点。最后，将各子片段保留的关键点按原始顺序合并，并去除重复的分裂点，得到简化轨迹。

图 3.13 展示了 DP 算法对轨迹采样点及折线形态的简化效果。容差决定了允许的几何近似误差：较大的容差通常保留较少的采样点，但可能舍弃局部转向细节；较小的容差保留更多细节，数据缩减程度也相应降低。此外，DP 仅依据空间几何误差选择关键点，并不直接控制速度变化和停留信息的损失；若下游任务依赖这些信息，还需采用相应指标评价压缩结果。

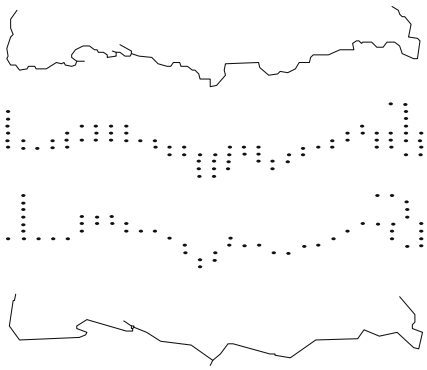


图 3.13 基于 Douglas-Peucker 算法压缩后的可视化轨迹

在 DP 算法的基础上，Hershberger-Snoeyink 算法[30]通过路径凸包（path hull）数据结构加速最大偏差点的查找。普通 DP 需要在每个递归片段中逐点计算距离；当分裂位置反复接近片段端部时，大量采样点会被重复扫描，最坏时间复杂度为 $O(n^2)$。该改进利用了凸包的几何性质：在论文采用的点到首尾连线所在直线的距离定义下，最大偏差点可在片段的凸包顶点中找到。因此，算法可通过凸包上的二分查找定位极值点，再比较距离确定分裂点，避免每次遍历全部采样点。

为支持递归过程，路径凸包以片段中部的顶点为标记点，分别维护其左、右子链的凸包，并通过历史栈记录构建时的更新操作。片段分裂后，算法通过撤销部分历史操作，复用包含标记点一侧的凸包结构，再为另一侧子片段重建结构。在原论文针对无自交折线的假设下，最远点查询与凸包维护的总开销为 $O(nlog\,n)$，从而改善了普通 DP 在不均衡分裂情况下的最坏运行时间。该方法的改进主要体现在计算效率，仍沿用 DP 的几何误差准则，并未额外控制时间信息或速度变化的损失。

（2）基于逐步合并的方法

基于逐步合并的轨迹压缩方法是传统轨迹压缩中的另一类常见方法。代表性算法包括 Bottom-Up 方法[31]和 SQUISH[32]等，其中 Bottom-Up 多用于离线轨迹简化（Trajectory Simplification），SQUISH 则更适用于在线轨迹压缩场景。

Bottom-Up 采用自底向上的贪心合并策略，从细粒度片段逐步形成较粗的分段表示。将这一思想用于轨迹简化时，设原始轨迹为 $T = \langle p_1, p_2, \ldots, p_n \rangle$，首先将每对相邻轨迹点连接为初始线段，再逐步合并相邻片段。

方法采用方向误差作为合并代价。记 $\theta_{i,j}$ 为从 p_i 指向 p_j 的方向角，取值范围为 $[0,2\pi)$。两个方向角之间的差异定义为：$\Delta(\alpha,\beta) = \min\{|\alpha-\beta|,\ 2\pi-|\alpha-\beta|\}$

对于轨迹片段 $T[i:j]$用首尾连线 $\overline{p_i p_j}$ 近似该片段时，其方向误差为：

$$\varepsilon(i,j) = \max_{i \le k < j} \triangle\left(\theta_{k,k+1}, \theta_{i,j}\right)$$

该式取原始片段中各线段与首尾连线之间的最大方向偏差。以下假设参与方向计算的线段长度非零。设当前分段序列为 $\langle S_1, S_2, \ldots, S_m \rangle$，其中相邻片段 $S_\mathrm{l} = T[i:j]$ 和 $S_{\mathrm{l}+1} = T[j:k]$ 共享端点 p_j。合并后以线段 $\overline{p_i p_k}$ 近似整个片段 $T[i:k]$，其合并代价定义为：$c_\mathrm{l} = \varepsilon(i,k)$。每轮迭代选择合并代价最小的一对相邻片段：$r = \arg\min_{1 \le \mathrm{l} < m} c_\mathrm{l}$。 若 $c_r \le \tau$，其中 τ 为预设方向误差阈值，则合并 S_r 和 S_{r+1}，删除两者之间的内部连接点，并重新计算新片段与左、右邻居的合并代价；其他候选代价保持不变。若最小合并代价也超过阈值，或仅剩一个片段，则停止合并。

图 3.14 给出了简化结果示意：原始轨迹包含 10 个采样点，简化轨迹保留 $(p_1$、p_3、p_6、$p_{10})$，分别以三条线段近似 $T[1:3]$、$T[3:6]$ 和 $T[6:10]$。图中的 $\varepsilon(\overline{p_i p_j})$ 对应上述 $\varepsilon(i,j)$。该方法每次选择当前代价最小的操作，不回溯已经完成的合并，因此不能保证获得全局最优结果。

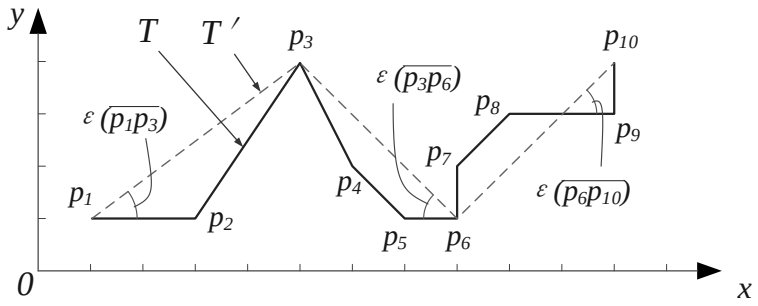


图 3.14 基于 Bottom-Up 的轨迹压缩

SQUISH 是一种面向 GPS 轨迹流的在线压缩方法，通过容量为 β 的缓冲区 B 保存当前保留点，并利用优先队列选择待删除点。与 Bottom-Up 合并相邻片段不同，SQUISH 在轨迹点持续到达时，根据删除各内部点的估计误差执行局部点删除。

设轨迹点为 $p_i = (x_i, y_i, t_i)$，其空间位置记为 $s_i = (x_i, y_i)$。对于缓冲区中的内部点 p_i，设其当前前、后两个保留点分别为 p_a和 p_b，满足 $t_a < t_i < t_b$。删除 p_i后，根据首尾点的时间与位置进行线性插值，得到时刻 t_i的估计位置：

$$\hat{s_i} = s_a + \frac{t_i - t_a}{t_b - t_a}(s_b - s_a)$$

点 p_i 的同步欧氏距离误差定义为：

$$e_i = d_{\mathrm{SED}}(p_i; p_a, p_b) = |s_i - \hat{s_i}|_2$$

与 DP 采用的空间垂直距离不同，SED 比较原始位置与同一时刻的插值位置。因此，即使轨迹接近直线，速度变化导致的位置偏差也能被反映出来。

算法以点的估计误差作为优先级数值 w_i，数值越小，越优先删除。内部点的优先级以其 SED 初始化；首点与当前末点的优先级设为 $+\infty$，以避免被删除。当新点到达时，将其加入缓冲区，并计算原末尾点的 SED。若需要腾出空间，则从当前内部点集合 B_{int}中选择：

$$p_r = \arg\min_{p_i \in B_{\text{int}}} w_i$$

删除 p_r 后，设其前、后两个保留点为 p_a 和 p_b ，将被删点的优先级分别累加到两个邻点：$w_a \leftarrow w_a + w_r, \quad w_b \leftarrow w_b + w_r$。随后更新优先队列的排序。这里，$w_i$ 在累加后是包含历史删除影响的误差估计，不能再简单视为该点当前的局部 SED。该机制通过局部更新减少重新评估的开销，但属于启发式估计；当保留点数过少、连续删除次数较多时，误差传播可能降低点排序的准确性。

（3）基于优化搜索的方法

基于优化搜索的方法将轨迹压缩建模为带约束的近似或关键点选择问题，通过动态规划、搜索等方式求解。不同方法采用不同的误差目标和预算约束，代表性算法包括 Bellman[33]和 Error-Search[34]等。

Bellman 利用动态规划求解曲线的分段线性近似问题：给定区间 $[a,b]$ 上的函数 $g(x)$ 和近似线段数 K，同时确定分割位置及各段的直线参数，使总平方近似误差最小。与 Bottom-Up 逐次确定局部合并不同，该方法通过比较不同分割位置，将整体优化问题分解为可递推求解的子问题。

首先，定义区间 $([u,v])$ 用一条直线近似时的最小误差：

$$h(u,v) = \min_{\alpha,\beta} \int_u^v [g(x) - (\alpha + \beta x)]^2 \, \mathrm{d}x$$

其中，α 和 β 分别为拟合直线的截距和斜率。对于给定区间，两者通过最小二乘拟合确定，$h(u,v)$ 因而表示该区间的最优单段近似代价。记 $F_k(v)$ 为在区间 $[a,v]$ 上采用 k 段直线近似时的最小总误差。仅使用一段直线时，有：$F_1(v) = h(a,v)$。当使用 $k \geq 2$ 段直线时，设最后一段的起点为 u。前面的区间 $[a,u]$ 由 $k-1$ 段直线近似，最后的区间 $[u,v]$ 由一段直线近似，因此得到递推关系：

$$\mathrm{F_k(v) = \min a \leq\ u \leq\ v\ \{F_{k-1}(u) + h(u,v)\}}$$

该式比较各候选分割位置对应的“前部最小误差”与“最后一段误差”之和，选择总误差最小的方案。计算实现时，可将分割位置离散为候选集合，依次计算各阶段的状态值，并记录

取得最小值的分割位置；最后从 $F_K(b)$ 回溯，恢复完整的分段方案。在采用离散候选集合时，所得最优结果限于该候选集合。

将这一思想用于离散轨迹时，可用采样点上的残差平方和替代积分误差，并以时间为自变量分别拟合位置分量。需要注意，各段由拟合直线表示，并不自动保证相邻线段连续连接或端点属于原始采样点；若应用要求保留原始关键点，还需相应限定候选端点与单段近似方式。

Error-Search 是一种求解方向误差最小化问题的精确算法。给定原始轨迹 $T = \langle p_1, p_2, \dots, p_n \rangle$ 和存储预算 W，算法从原始采样点中选择不超过 W 个点，并保留首尾点，构成简化轨迹 $T' = \langle p_{s_1}, \dots, p_{s_m} \rangle$，其中 $1 = s_1 < \cdots < s_m = n$。简化轨迹的误差取各近似线段方向误差的最大值：

$$\varepsilon(T') = \max_{1 \le k < m} \varepsilon(s_k, s_{k+1})$$

因此，记 F_W 为保留首尾点且点数不超过 W 的所有合法简化轨迹组成的集合，Min-Error 问题的目标为：

$$\varepsilon^* = \min_{T' \in F_W} \varepsilon(T')$$

Error-Search 首先构造候选误差集合：

$$\mathcal{E} = \{\varepsilon(i, j) \mid 1 \le i < j \le n\}$$

由于任意简化轨迹的误差均等于其中某条近似线段的误差，最优值 ε^* 必然属于 $\mathcal{E}$。算法因而只需在这一有限集合中搜索，无需枚举所有关键点组合。

为加快候选误差的计算，方法利用了方向角的几何性质：与近似线段方向偏差最大的原始线段，其方向最接近该近似线段的反方向。对于固定起点，算法随终点向后移动，增量维护原始线段方向的有序集合，再通过二分查找定位最接近反方向的方向角，从而避免对每个候选区间重新逐段扫描。全部候选误差可在 $O(n^2 \log n)$ 时间内生成，随后按升序排列。

对于候选阈值 η，记 F_η 为方向误差不超过 η 的所有合法简化轨迹组成的集合。算法调用 Min-Size 精确求解算法，计算该集合中轨迹所需保留的最少点数：

$$M(\eta) = \min_{T' \in F_\eta} |T'|$$

若 $M(\eta) \le W$，则该阈值满足预算要求，可以继续搜索更小的候选误差；否则，说明阈值过严，需要搜索更大的候选误差。该判断具有单调性：一个阈值可行时，比它更大的阈值也必然可行。因此，算法对排序后的 $\mathcal{E}$ 进行二分搜索，最终返回最小可行误差及对应的简化轨迹。与预先指定误差阈值的方法不同，Error-Search 以保留点数为约束，求得该预算下

的最小方向误差。其计算开销主要来自候选集合构造和多次精确可行性检查，保存候选集合需要 $O(n^2)$ 空间，处理长轨迹时需考虑这一存储开销。

3.4.3. 基于深度学习的轨迹压缩

深度学习方法通过训练模型学习轨迹的紧凑表示或关键点选择策略。其中，表示压缩方法将轨迹编码为低维潜在表示，并通过解码重构评价信息保留程度；轨迹简化方法则学习从原始序列中选择保留点。本节分别介绍基于自编码器的轨迹压缩、基于强化学习的轨迹简化，以及基于自监督重构的轨迹简化，代表方法为 CVAE[39]、RLTS[40]和 S3[42]。

通常情况下，该类方法首先需对原始轨迹进行表示转换，例如将连续 GPS 点映射为离散网格编号，或将其空间位置、时间信息及运动属性转化为嵌入向量。随后，模型利用循环神经网络[35]、序列到序列（Sequence-to-Sequence，Seq2Seq）模型[36]、自编码器[37]或 Transformer[38]等结构对序列进行编码，获取固定或较低维度的轨迹表示。该表示既可作为压缩后的表达形式，以减少原始序列带来的存储与计算开销，也能进一步服务于下游任务。

（1）基于自编码器的方法

基于自编码器的方法由编码器将轨迹序列映射为低维潜在表示，再由解码器重构位置序列，通过重构误差约束压缩过程。与直接删除采样点的方法不同，其压缩结果是潜在表示，需要配合解码器恢复轨迹。下面以采用卷积变分自编码器的车辆轨迹压缩方法为例，介绍其编码、潜在变量与解码过程。

Rákos 等[39]采用卷积变分自编码器（Convolutional Variational Autoencoder，CVAE）压缩车辆轨迹，其编码与解码流程如图 3.15 所示。方法以持续 6 秒、包含 60 个时间步的车辆轨迹为输入，每个时间步包含横向和纵向两个位置分量。预处理时，将各条轨迹的起点平移至原点，得到输入矩阵 $X \in R^{2\times 60}$。

编码器沿时间轴利用一维卷积提取局部运动特征，并通过池化缩短序列，随后由两个分支分别输出潜在分布的均值与标准差。用统一符号表示，编码分布为：

$$q_\phi(z \mid X) = \mathcal{N}\left(\mu_\phi(X), \mathrm{diag}\left(\sigma_\phi^2(X)\right)\right)$$

其中，ϕ 为编码器参数，$\mu_\phi(X)$ 和 $\sigma_\phi(X)$ 均为 d 维向量，论文比较了 3 至 10 维潜在空间。与普通自编码器直接输出确定性向量不同，CVAE 将输入映射为潜在空间中的高斯分布，再从中采样上下文向量。该采样可写为：

$$z = \mu_\phi(X) + \sigma_\phi(X) \odot \xi, \qquad \xi \sim \mathcal{N}(0, I)$$

其中，$\odot$ 表示逐元素相乘。解码器接收 d 维向量 z，通过全连接层、转置卷积与自适

应平均池化恢复序列长度，输出重构轨迹：

$$\hat{X} = f_\theta(z), \qquad \hat{X} \in R^{2\times 60}$$

其中，θ 为解码器参数。模型训练同时考虑位置重构误差、相邻时间步变化的重构误差，以及潜在分布的正则化。对一次采样，其损失可概括为：

$$L = \mathrm{MSE}(X, \hat{X}) + 0.1\,\mathrm{MSE}(DX, D\hat{X}) + \lambda D_{\mathrm{KL}}\left(q_\phi(z \mid X) \,|\, \mathcal{N}(0, I)\right)$$

其中，MSE 表示均方误差，D 表示沿时间轴的一阶差分，λ 为正则化权重。第一项约束重构位置接近原始位置；第二项比较相邻时间步的位移变化，减少重构轨迹中的局部波动，论文将其权重设为 0.1；第三项约束编码分布接近标准正态先验，使潜在表示在保留重构信息的同时受到分布正则化。

潜在维度 d 决定了表示的紧凑程度：较小的维度减少表示所需的数值数量，但也限制重构能力，论文通过不同潜在维度下的实验评价这一关系。模型训练无需人工类别标注，学到的潜在表示还可用于车辆操纵识别。论文以该表示训练支持向量机，区分车道保持、左变道和右变道，说明压缩后的向量仍保留了可用于行为分析的信息。

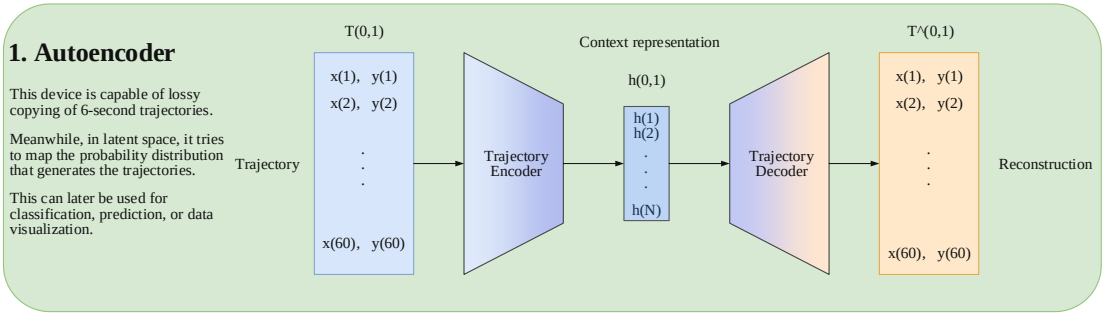


图 3.15 CVAE 的框架图（改绘自[39]）

（2）基于强化学习的方法

基于强化学习的轨迹简化方法将点选择视为序列决策问题，通过状态描述当前轨迹，通过动作执行点删除或跳点，并利用简化误差构造奖励，训练点选择策略。下面以 RLTS 及其变体介绍这一过程。

在此类方法中，RLTS[40]是具有代表性的轨迹压缩方法。该方法将轨迹简化建模为马尔可夫决策过程[41]。给定原始轨迹 $T = \langle p_1, p_2, \ldots, p_n \rangle$ 和存储预算 W，算法先将前 W 个点存入缓冲区。此后，每当新点到达，便从已有点中选择一个点删除，为新点腾出空间。与直接删除最小优先级点的方法不同，RLTS 先筛选候选点，再通过学习得到的策略选择删除对象。

为计算点删除的局部影响，可在概念上将新点追加到缓冲区，记当前序列为

$\langle q_1, \ldots, q_{W+1} \rangle$，首点与新到达的末点不参与删除。以 PED 为例，内部点 q_j 的估计值定义为：

$$v(q_j) = d_{\text{PED}}(q_j, \overline{q_{j-1}q_{j+1}}), \qquad 2 \le j \le W$$

该值表示删除 q_j、以其前后点连线替代原有两段时，该点产生的空间偏差。算法按估计值升序选取前 k 个候选点，记为 $c_1, \ldots, c_k$，构成状态：

$$s_t = \big(v(c_1), v(c_2), \ldots, v(c_k)\big)$$

其中，$k \le W - 1$ 为超参数，使状态维度不随缓冲区容量变化。动作 $a_t = j$ 表示删除候选点 c_j。策略网络根据当前状态输出各动作的概率：

$$\pi_\theta(a_t = j \mid s_t) = [\text{softmax}(As_t + b)]_j,$$

其中，$\theta = \{A, b\}$ 为可学习参数。算法根据该分布采样动作，因此不一定删除估计值最小的点。删除后，分别更新两个邻点的估计值，更新时同时考虑邻点自身和刚被删点相对于新近似线段的误差，取两者的较大值。新点加入后，再计算原末尾点的估计值，据此构造下一状态。

模型的奖励由动作前后的简化误差之差确定。记动作前后的简化轨迹分别为 T_t' 和 T_t''，其误差相对于对应的原始轨迹前缀计算，则：

$$r_t = \varepsilon(T_t') - \varepsilon(T_t'')$$

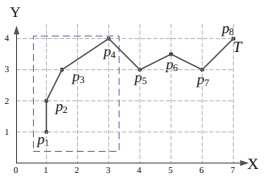
动作引起的误差增量越小，奖励越大。在初始误差为 0 且不折扣未来奖励时，各步奖励相加可消去中间项，得到：

$$\sum_t r_t = -\varepsilon(T_{\text{final}})$$

因此，最大化累计奖励与最小化最终简化误差具有一致的目标。论文采用策略梯度方法训练策略网络；奖励计算用于离线训练，在线简化时根据当前状态调用已训练的策略。

RLTS-Skip 在上述 k 个删除动作之外，增加 J 个跳点动作。选择跳过 j 个点时，直接丢弃当前待处理点及其后连续的 $j-1$ 个点，再继续扫描，不将这些点逐一加入缓冲区。因此，其动作空间包含 $k + J$ 个动作；当 $J = 0$ 时退化为 RLTS。跳点能够减少状态构造和策略调用次数，参数 J 则用于控制效率与简化误差之间的权衡。

图 3.16 给出了 $W = 4$ 时的 RLTS-Skip 执行过程。算法先保存 $\langle p_1, p_2, p_3, p_4 \rangle$。处理 p_5 时，虽然 p_2 的估计值 0.243 小于 p_3 的 0.354，策略仍选择删除 p_3，说明其决策并非简单取最小估计值。随后，策略跳过 p_6 和 p_7，在处理 p_8 时删除 p_2，最终得到 $\langle p_1, p_4, p_5, p_8 \rangle$，PED 误差为 0.693。



Initial	Store < p_1, p_2, p_3, p_4 > into the buffer with the initial reward 0.0				
Point	State	Action	Buffer	Reward	Accum Reward
p_5	s_1 = {$v(p_2)$ = 0.243, $v(p_3)$ = 0.354, $v(p_4)$ = 1.0}	Drop p_3	< p_1, p_2, p_4, p_5 >	-0.354	-0.354
p_6	s_2 = {$v(p_2)$ = 0.693, $v(p_5)$ = 0.728, $v(p_4)$ = 1.265}	Skip p_6 and p_7	< p_1, p_2, p_4, p_5 >	-0.278	-0.632
p_7	-			-	-
p_8	s_3 = {$v(p_2)$ = 0.693, $v(p_5)$ = 1.0, $v(p_4)$ = 1.265}	Drop p_2	< p_1, p_4, p_5, p_8 >	-0.061	-0.693
Output	Return T' = < p_1, p_4, p_5, p_8 > with $\varepsilon(T')$ = 0.693				

图 3.16 RLTS 轨迹压缩过程示意图（改绘自[40]）

在批处理模式下，原始点始终可以访问，因而能够构造信息更充分的状态。RLTS+在计算候选点误差时纳入已扫描但已删除的点；RLTS++则将完整轨迹放入缓冲区，逐次删除点直至满足预算 W。相应的 Skip 变体沿用这些状态构造方式。这些方法利用更多原始数据辅助决策，但也增加了候选点误差计算与状态维护的开销。

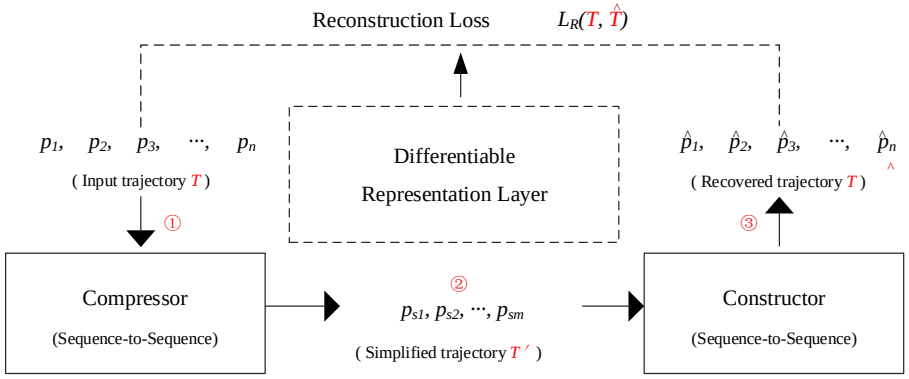


图 3.17 S3 的架构图（改绘自[42]）

（3）基于自监督重构的方法

基于自监督重构的轨迹简化方法，以原始轨迹自身作为训练目标，通过重构任务学习关键点选择，无需预先提供人工标注的简化结果。S3[42] 采用序列到序列再到序列（Seq2Seq2Seq）框架，由 Compressor 和 Constructor 两个相连的 Seq2Seq 模型组成。如图 3.17 中的①—③所示，Compressor 接收原始轨迹 $T = \langle p_1, \ldots, p_n \rangle$，生成包含 m 个点的简化轨迹 $T' = \langle p_{s_1}, \ldots, p_{s_m} \rangle$，其中 $1 = s_1 < \cdots < s_m = n$；Constructor 再根据 T' 输出重构轨迹 $\hat{T}$。这一过程可表示为：

$$T' = C_\phi(T, m), \qquad \hat{T} = R_\psi(T')$$

其中，ϕ 和 ψ 分别为两个模型的参数，目标点数 m 用于控制简化规模。

输入模型前，S3 将研究区域划分为不相交的网格，把位置 p_i 映射为网格标记 v_i，并利用 Skip-gram 初始化网格的向量表示。Compressor 通过双向循环网络编码位置序列，再利用注意力机制结合输入序列与当前解码状态计算候选位置的输出分布。解码过程中还引入剩余输出长度 $m - t$，使模型能够根据指定的目标点数生成简化序列，并对已选位置进行标记，避免重复选择。

由于位置选择属于离散操作，若直接将选中的网格标记传给 Constructor，重构损失难以通过该操作反向传播至 Compressor。为此，图中的可微分表示层采用 Gumbel–Softmax 连续松弛。记网格集合为 V，网格 v 的嵌入为 e_v，第 t 个输出位置的松弛选择权重为 $\alpha_{t,v}$，则其连续表示为：

$$\widetilde{g_t} = \sum_{v \in V} \alpha_{t,v}\, e_v, \qquad \sum_{v \in V} \alpha_{t,v} = 1$$

其中，$\alpha_{t,v}$ 根据 Compressor 的输出得分加入 Gumbel 噪声后计算得到。前向计算时，模型采用离散选择得到实际位置；反向传播时，则利用上述连续表示计算梯度。这样，Constructor 的重构误差便能反向影响 Compressor 的选点参数，而不只是训练重构模型本身。

Constructor 读取简化序列的嵌入，输出原始长度的位置标记序列。论文采用负对数似然作为重构损失。记 $\widehat{v_i}$ 为第 i 个重构位置对应的网格标记，$\tilde{G} = \langle \widetilde{g_1}, \ldots, \widetilde{g_m} \rangle$ 为可微分层传递的表示序列，则损失可概括为：

$$L_R(T, \hat{T}) = -\sum_{i=1}^{n} \log P_\psi \left(\widehat{v_i} = v_i \mid \tilde{G} \right)$$

其中，$P_\psi\left(\widehat{v_i} = v_i \mid \tilde{G} \right)$ 表示 Constructor 在第 i 个位置为真实网格标记赋予的概率。最小化该损失，会促使 Compressor 选择有助于恢复原始位置序列的点。这里比较的是网格标记的重构概率，并非原始坐标与重构坐标之间的均方误差。

在上述基础框架之外，论文进一步引入上下文感知图，以利用多条轨迹中的移动关联。图中每个节点对应一个网格；若轨迹中存在两个网格之间的连续移动片段，则在相应节点之间连边，边权为包含该转移片段的轨迹数量。随后，通过图卷积网络聚合邻域信息，学习包含跨轨迹移动关联的网格嵌入，并将其用于序列表示。该扩展利用原始轨迹构图，不要求预先提供道路网络或 POI 标签。

S3 的优势在于，利用原始轨迹即可构造重构监督，简化过程也无需反复计算 PED、SED 或 DAD 来决定每个点的去留。训练后，Compressor 输出的 T' 即为简化结果，Constructor 主要承担重构监督的作用。不过，重构损失与这些几何误差指标并不相同，因此不能保证所得轨迹在各项误差指标上均达到最优；其效果仍需通过相应误差指标及下游任务进行评价。

3.5 轨迹恢复

轨迹采集受到设备采样策略、信号遮挡及观测设备覆盖范围等因素的影响，已有记录往

往只能反映移动过程中的部分位置，难以直接确定观测之间的行驶过程。轨迹恢复利用这些有限观测及相关辅助信息，推断缺失位置或可能经过的路径，为后续轨迹分析提供更完整的数据表示。

不同恢复方法利用的信息和推断依据有所差异：几何插值依据局部运动假设估计位置，路网搜索与概率推断结合道路连接关系和观测匹配程度选择路径，深度学习方法则利用训练数据中的时空模式辅助恢复。本节围绕位置序列恢复与道路路径恢复，介绍相关问题定义、约束条件及评价方式，并分别讨论传统方法与深度学习方法的推断机制和适用条件。

3.5.1. 轨迹恢复概述

轨迹恢复根据稀疏或不完整的观测，推断未观测时段的位置或行驶路径。设 GPS 观测轨迹为 $T_{obs} = \langle p_1, \dots, p_n \rangle$，其中 $p_i = (x_i, y_i, t_i)$ 记录空间位置与时间戳，且 $t_1 < \cdots < t_n$。相邻观测的时间间隔可以不相等，位置也可能包含测量误差。对于交通摄像头数据，观测则可表示为车辆在特定时刻经过某一路段的记录。恢复过程除利用已有观测外，还可结合道路网络、历史轨迹及环境信息。

根据输出形式，轨迹恢复主要包括位置序列恢复和道路路径恢复。位置序列恢复在指定时刻估计移动对象的位置。例如，给定目标采样间隔 Δt，输出可表示为：

$$\hat{T} = \langle (\widehat{s_1}, \tau_1), \dots, (\widehat{s_m}, \tau_m) \rangle, \qquad \tau_j = t_1 + (j-1)\,\Delta t$$

其中，m 为目标时间序列的点数，$\widehat{s_j}$ 为时刻 τ_j 的估计位置，目标时刻位于观测覆盖的时间范围内。当目标采样间隔小于原始观测间隔时，模型需要估计观测之间的位置；对于含噪观测，也可能同时校正已有位置。因此，恢复既可用于补充漏采数据，也可用于将低采样率轨迹细化为高采样率序列。

道路路径恢复则以有序路段序列 $\hat{R} = \langle e_1, \dots, e_K \rangle$ 为输出，推断相邻观测之间可能经过的道路。其中，相邻路段应满足道路连接关系、行驶方向与通行限制，并与观测对应的道路位置衔接。该输出确定了可能的行驶路线，但并不直接给出车辆在各个缺失时刻的位置；若需进一步恢复位置序列，还要估计沿路径的运动过程。

轨迹恢复的主要困难是观测不足导致的多解性：同一对观测位置之间可能存在多条可行路径，同一路径也可能对应不同的速度变化过程。因此，恢复结果是给定观测与模型条件下的估计，不能仅凭路径连通或点数增加就认定其准确。评价时，应根据输出形式选择指标：位置序列恢复关注对应时刻的位置误差，道路路径恢复关注路段与行驶路线的匹配程度，并结合时间、运动及通行约束检查结果的合理性。

3.5.2. 传统轨迹恢复

传统轨迹恢复可分为几何插值、路网路径搜索和候选状态联合推断三种思路。它们分别侧重局部运动假设、道路可达性与观测不确定性，所需输入和输出形式也有所区别。地图匹配为后两者提供观测到道路的关联，恢复还需根据目标补全中间道路或位置。

（1）基于几何插值的方法

几何插值方法主要依据相邻观测点之间的空间位置和时间关系对缺失轨迹进行补全。线性插值法假设移动对象在相邻观测点之间以匀速直线运动，并按照时间比例在两点之间插入新的轨迹点；样条插值等方法则通过构造平滑曲线恢复连续运动过程。该类方法实现简单、计算效率较高，适合采样间隔较短且运动变化较平滑的场景。但由于其主要依据几何位置关系进行恢复，通常难以考虑道路连通性、交通规则和实际可达路径，因此在车辆轨迹恢复中可能产生偏离道路或穿越不可达区域的结果。

（2）基于路网路径搜索的方法

路网路径搜索方法将道路网络表示为图 $G=(V,E)$，其中节点表示道路交叉口等连接位置，边表示道路段。首先，将相邻 GPS 观测点关联到候选路网位置；随后，根据道路连接关系、行驶方向和通行限制确定可行路径，并利用 Dijkstra 算法[43]、A*搜索算法[44] 等方法选择代价最小的路径。设两个候选位置对应节点 u 和 v，合法路径集合为 $\Omega(\mathrm{u},\mathrm{v})$，则搜索目标为：

$$P^{*}=\arg\min_{P\in\Omega(u,v)}\sum_{e\in P}w(e)$$

其中，$w(e)$ 为道路段 e 的通行代价。采用道路长度作为边权时，得到距离最短的路径；采用预计行驶时间作为边权时，得到预计耗时最少的路径。搜索结果由有序道路段组成，可用于补全两次观测之间缺失的道路序列。

该方法能够将恢复范围限制在合法路网上，但所得结果仅是在指定边权下的最优路径，不一定对应车辆真实经过的路线。尤其在采样间隔较长、存在多条可行路线时，仅依靠路径长度或预计耗时，仍难以充分处理定位误差和路径选择的不确定性，需要进一步结合观测匹配度与时空约束进行联合推断。

（3）基于候选状态联合推断的方法

基于候选状态联合推断的方法，为各 GPS 观测点生成多个候选道路位置，结合观测匹配度与道路间的转移关系，联合选择与观测序列相符的道路路径。代表方法包括基于隐马尔可夫模型（Hidden Markov Model，HMM）的地图匹配[2]和 ST-Matching[45]等。

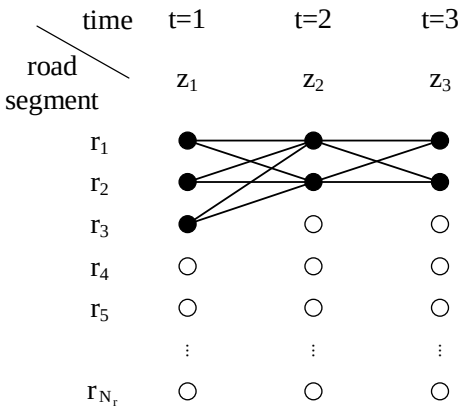


图 3.18 HMM 轨迹恢复中的候选道路状态序列

Newson 和 Krumm[2]提出的 HMM 地图匹配方法，将道路段视为隐状态，将带有定位误差的 GPS 位置视为观测。设观测序列为 $Z = \langle z_1, \ldots, z_n \rangle$，道路段集合为 $\{r_1, \ldots, r_{N_r}\}$。对于观测 z_t 与候选道路段 r_j，以该路段上距离观测最近的位置 $x_{t,j}$ 作为候选匹配位置。发射概率 $b_t(j) = P(z_t \mid r_j)$ 衡量该道路段与当前观测的匹配程度，原文采用高斯噪声模型，根据观测到候选位置的距离计算，具体原理见第 2.1.2 节。

图 3.18 展示了按观测时刻展开的候选状态图。每一列对应一次 GPS 观测，每一行对应一条道路段。黑点表示参与推断的候选状态，空心点表示因距离观测过远而被排除的状态。例如，第一次观测对应 r_1、r_2、r_3 三个候选状态，后两次观测分别保留 r_1、r_2 两个候选状态。相邻列之间的连线表示候选位置之间可能发生的道路转移，连接路径可以经过多条中间道路，并不要求两个候选道路段直接相邻。

为评价一次转移，算法先搜索候选位置 $x_{t,i}$ 与 $x_{t+1,j}$ 之间的最短道路路径，再比较其长度与相邻 GPS 观测之间的地表距离：

$$\Delta_t(i,j) = \left| d_G(x_{t,i}, x_{t+1,j}) - d_{\text{geo}}(z_t, z_{t+1}) \right|$$

其中，d_G 表示道路最短路径长度，d_{geo} 表示地表大圆距离。两者差异越大，该转移越可能包含不合理绕行。原文采用指数分布评价这一差异，转移权重可写为：

$$a_t(i,j) \propto \exp\left(-\frac{\Delta_t(i,j)}{\beta} \right)$$

其中，$\beta > 0$ 控制对距离差异的容忍程度。不可达或经合理性检查被排除的转移不参与后续路径选择。

算法随后通过维特比动态规划联合选择候选状态。记 $\delta_t(j)$ 为处理前 t 个观测、且以道路段 r_j 结束的状态序列的最大评分，则初始化为 $\delta_1(j) = b_1(j)$，递推关系为：

$$\delta_t(j) = b_t(j) \max_i \{\delta_{t-1}(i) a_{t-1}(i,j)\}$$

计算时同时记录取得最大值的前驱状态，处理完全部观测后，从评分最高的终止状态回溯，得到最优道路状态序列。这一过程联合考虑各观测的匹配程度和前后道路转移的合理性，减少仅依据单个观测选择最近道路造成的错误。

用于轨迹恢复时，将选定状态之间对应的道路路径依次连接，即可补全观测间隔内缺失的中间道路。这里恢复的是道路行驶路径，并不直接给出每个缺失时刻的精确位置。若候选道路中遗漏真实道路，或稀疏观测不足以区分多条可行路线，最终路径仍可能存在偏差。

ST-Matching[45] 是一种面向低采样率 GPS 轨迹的全局地图匹配方法。如图 3.19 所示，其流程分为候选生成、时空分析和结果匹配三个阶段，分别对应图中的左、中、右三个模块。

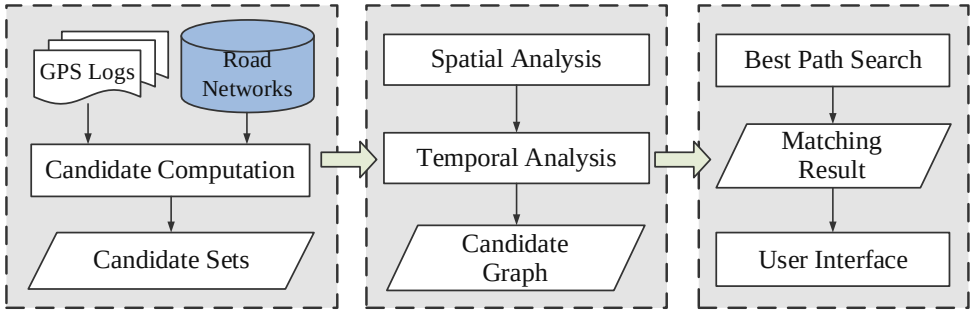


图 3.19 ST-Matching 轨迹恢复方法流程（改绘自[45]）

图左侧的候选生成模块接收 GPS 轨迹与道路网络，在每个观测点附近搜索候选道路段，并将观测点投影到这些道路段上，形成候选点集合。设第 i 个观测点的候选位置为 $c_i^1, c_i^2, \ldots$ 每个观测点通常对应多个候选位置，算法需要结合前后观测，从各候选集合中联合选择匹配结果。

图中部依次进行空间分析和时间分析。空间分析考虑两个因素：一是观测点与候选位置之间的距离，采用高斯模型计算观测评分 $N_i(b)$；二是候选位置之间的路径绕行程度，以相邻 GPS 观测的直线距离与候选最短道路路径长度之比作为传递评分 $V_i(a,b)$。其中，a 和 b 分别表示前、后两个观测时刻的候选编号。对于相同的观测点对，候选道路路径越长，传递评分越低。

时间分析利用候选路径长度与观测时间间隔计算平均速度，再将道路段的典型速度向量与重复该平均速度组成的向量进行余弦相似度计算，得到评分 $H_i(a,b)$。三项评分相乘，形成从候选点 c_{i-1}^a 到 c_i^b 的综合评分：

$$F_i(a,b) = N_i(b)V_i(a,b)H_i(a,b)$$

时空分析的输出是图中的候选图（Candidate Graph）。图中节点对应各观测点的候选位置，边对应相邻候选位置之间的最短道路路径，并携带上述综合评分。

图右侧的最优路径搜索模块在候选图中寻找累计评分最高的序列。记 $f_i(b)$ 为处理前 i

个观测、且以候选点 c_i^b 结束的最大累计评分，则：

$$f_i(b) = \max_a \{f_{i-1}(a) + F_i(a,b)\}$$

其中，a 遍历前一观测时刻的候选点，初始评分为 $f_1(b) = N_1(b)$。算法逐层计算并记录最优前驱，最后回溯得到匹配序列，将相邻选定候选点之间的道路路径连接为结果。图中的 Matching Result 表示这一输出，User Interface 则用于展示匹配结果。

该方法的主要计算开销来自候选位置之间的最短路径搜索，候选数量增加会带来更多路径计算。此外，原文的时间评分采用余弦相似度，其中正的平均速度会被约去，因此不能直接将该项解释为严格的行驶时间或超速检查。

3.5.3. 基于深度学习的轨迹恢复方法

基于深度学习的轨迹恢复方法通过学习观测序列、道路结构及历史行为，为缺失位置或路径推断提供依据。本节选取 MTrajRec[46]、ImIn-GAIL[48]、RNTrajRec[47]和 TrajRecovery[49]作为代表，分别介绍序列生成、对抗式模仿学习、路网增强 Transformer 及任务驱动型恢复，重点说明其输入输出、模型结构与恢复过程。

（1）基于序列生成的方法

序列生成类方法将轨迹恢复建模为从稀疏观测序列到目标采样间隔下完整轨迹序列的映射，通过编码器提取观测轨迹的时空特征，再由解码器逐步生成恢复结果。MTrajRec[46]采用多任务 Seq2Seq 框架，将缺失位置恢复与地图匹配联合建模，直接输出道路约束下的高采样率轨迹，减少先恢复自由空间坐标、再进行地图匹配的分阶段处理。

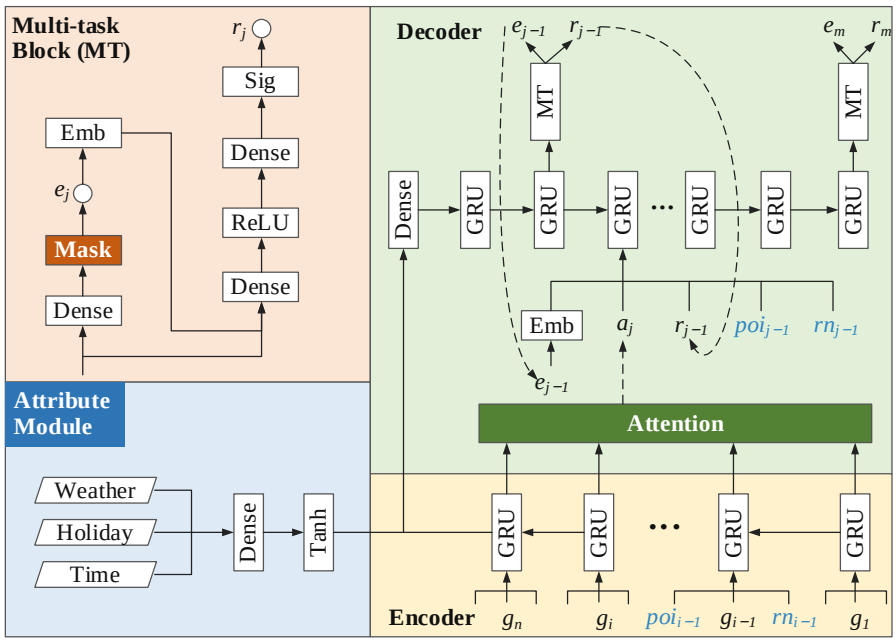


图 3.20 MTrajRec 轨迹恢复模型结构（改绘自[46]）

如图3.20所示，模型主要包括编码器（Encoder）、注意力层（Attention）、解码器（Decoder）、多任务模块（Multi-task Block）和属性模块（Attribute Module）。设输入轨迹为 $T = \langle p_1, \dots, p_n \rangle$，目标采样间隔为 Δt。模型将各 GPS 点转换为网格位置与时间索引组成的输入 $g_i = (x_i, y_i, \mathrm{tid}_i)$，其中 x_i 和 y_i 为网格坐标，时间索引为：

$$\mathrm{tid}_i = \left\lfloor \frac{t_i - t_0}{\Delta t} \right\rfloor$$

其中，t_0 为时间参考起点。时间索引将稀疏观测对应到目标时间序列，使模型能够区分已观测时刻与待恢复时刻。图下方的编码器利用 GRU 处理该输入序列，提取观测点之间的时空依赖关系。解码时，图中间的注意力层对编码器各时刻的输出进行加权，形成当前生成步骤的上下文向量，使恢复过程能够利用不同观测点的信息。

图右上方的解码器按照目标采样间隔逐步生成轨迹点。与直接预测经纬度不同，模型将第 j 个恢复点表示为：

$$\hat{a}_j = (\hat{e}_j, \hat{r}_j, t_j), \qquad 0 \le \hat{r}_j \le 1$$

其中，$\hat{e}_j$ 为预测路段编号，$\hat{r}_j$ 为从该路段起点到预测位置的沿路距离占路段总长度的比例，t_j 为目标时间序列中的对应时刻。因此，确定路段及段内比例后，即可确定该时刻在路网上的位置。

图左上方的多任务模块具体完成这两项预测。解码器的隐藏状态首先经过全连接层和带约束掩码的 Softmax，得到路段概率分布，并选择概率最大的路段；随后，将所选路段的嵌入与解码器隐藏状态拼接，经全连接层及 Sigmoid 函数预测段内比例。该串联结构将所选路段的嵌入作为比例预测的输入，使段内位置估计显式依赖路段选择结果。当前预测的路段嵌入与段内比例还会作为下一步解码的输入。

其中，Mask 层用于补充网格化过程中损失的精确位置信息。在已有 GPS 观测对应的输出时刻，模型依据观测点与各路段投影位置之间的距离，对路段预测概率进行加权，并将距离达到 50 m 的候选路段权重置零；在缺失时刻，各路段的掩码权重均设为 1，不施加这一直接距离约束。因此，掩码主要限制已有观测附近的匹配范围，缺失位置仍需结合序列上下文进行推断。

图左下方的属性模块用于引入环境信息。天气、节假日和小时等特征经过编码与全连接层处理后，与编码器末端隐藏状态融合，共同初始化解码器。对于随位置变化的空间属性，如不同类别的 POI 密度、道路交叉口数量和路段等级，模型则在编码与解码的各时间步分别输入，对应图中的 poi 和 rn 特征。

训练时，模型利用高采样率地图匹配轨迹提供监督，分别采用交叉熵和均方误差约束路段分类与段内比例预测。用统一符号概括，其联合损失为：

$$L = L_{\mathrm{CE}} + \lambda L_{\mathrm{MSE}}$$

其中，L_{CE} 衡量预测路段分布与真实路段之间的差异，L_{MSE} 衡量预测比例与真实比例之间的差异，λ 用于平衡两项任务。该方法的优势在于将路段选择与段内定位联合学习，既输出道路归属，也恢复指定时刻的细粒度位置。但恢复点位于路网上，并不等于预测路段序列必然连通，其恢复质量仍取决于路段分类、位置比例预测及训练数据的覆盖情况。

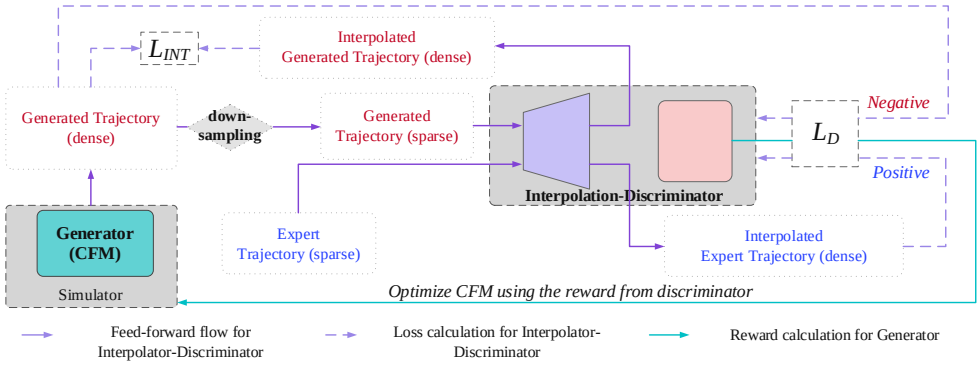


图 3.21 ImIn-GAIL 轨迹恢复框架（改绘自[48]）

（2）基于对抗式模仿学习的方法

基于对抗式模仿学习的方法通过区分专家行为与模型生成行为，为驾驶策略学习提供反馈。在专家轨迹稀疏时，仅利用已有观测进行判别，难以约束未观测时段的驾驶行为。在此类研究中，ImIn-GAIL[48]将轨迹插值引入生成对抗模仿学习框架，联合学习缺失驾驶信息与专家行为策略，其结构如图 3.21 所示。

图左侧的生成器在仿真环境中执行驾驶策略 $\pi_\theta(a_t \mid s_t)$，根据当前状态 s_t 选择动作 a_t，并通过连续交互生成稠密驾驶轨迹。这里的轨迹点包含驾驶状态与动作信息，而非仅包含空间坐标。其中，状态描述车辆速度、与前车的相对距离、交通信号等信息，动作控制车辆的加减速行为。因此，模型学习的是交通环境下的驾驶响应关系。

图上方的下采样器将生成的稠密轨迹转换为稀疏轨迹，并保留原始稠密数据作为插值监督。对于下采样后相邻的两个观测时刻 t_s 和 t_e，从原稠密轨迹中选取区间内的时刻 t，即可构造“两端观测—中间驾驶信息”的训练样本。插值器分别将两端的状态与动作编码为向量，再与时间偏移 $t - t_s$ 拼接，通过多层感知机预测中间时刻的状态与动作。图中的 L_{INT} 衡量插值结果与生成轨迹中对应稠密样本之间的差异，使插值模块能够在缺少真实稠密专家数据的条件下获得监督。

图右侧的插值—判别器包含相互连接的插值模块和判别模块。插值模块利用上述训练样

本学习稀疏到稠密的映射，并将同一映射用于图下方的稀疏专家轨迹，得到插值后的稠密专家数据。判别模块则以这些专家数据为正样本，以生成器产生的稠密驾驶数据为负样本，判断输入的状态—动作对是否具有专家行为特征。插值—判别器的联合训练目标可概括为：

$$L = \lambda L_{\mathrm{INT}} + (1-\lambda) L_D$$

其中，L_D 为区分专家数据与生成数据的判别损失，$0 \le \lambda \le 1$ 用于平衡插值与判别两项任务。图中的紫色实线表示数据前向传递，紫色虚线对应插值损失与判别损失的计算。

判别结果还通过图下方的青色反馈路径用于更新生成器。记 $D_{\psi}(s_t, a_t)$ 为判别器将当前状态—动作对判断为专家行为的概率，则生成器使用的代理奖励为：

$$\widetilde{r_t} = -\log\left(1 - D_{\psi}(s_t, a_t)\right)$$

当生成行为更接近专家行为、判别得分提高时，相应奖励也增大。模型据此采用信赖域策略优化（TRPO）更新驾驶策略，并交替训练生成器与插值—判别器。由此，生成轨迹为插值学习提供稠密监督，插值后的专家数据为行为判别补充未观测时段的信息，判别反馈再推动生成策略接近专家策略。

该方法的优势在于将插值纳入行为学习过程，使插值模块能够随策略学习持续更新，减轻固定插值预处理产生的偏差对后续学习的影响。不过，其目标主要是从稀疏专家轨迹学习可用于交通仿真的驾驶策略，不能直接等同于对任意 GPS 轨迹的逐点还原；其效果也受到仿真环境和生成轨迹质量的影响。

（3）基于路网增强 Transformer 的方法

基于 Transformer 的轨迹恢复方法利用自注意力机制建立不同观测点之间的联系，为缺失位置推断提供序列上下文。RNTrajRec[47]进一步将道路子图引入轨迹编码过程，使观测点表示同时包含周围道路结构和前后观测信息。如图 3.22 所示，其框架由右下方的 GridGNN、左上方的 GPSFormer 以及右上方的多任务解码器组成，分别负责路网表示、轨迹编码和位置恢复。

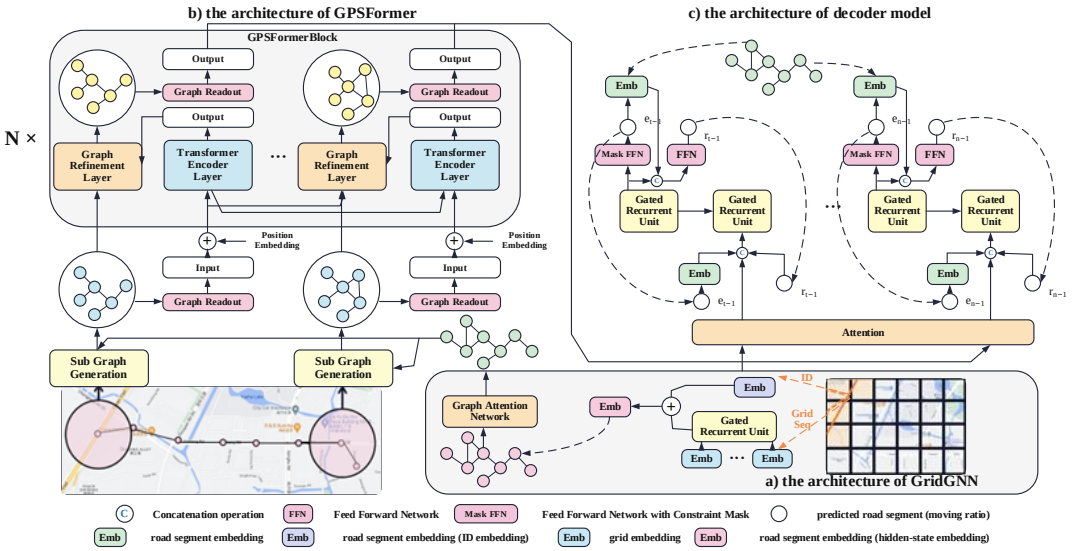


图 3.22 RNTrajRec 轨迹恢复模型结构（改绘自[47]）

图中的 GridGNN 首先将研究区域划分为等大小网格，把每条路段表示为其经过的网格序列。模型利用 GRU 编码该序列，并与路段 ID 嵌入融合，得到初始路段表示；随后，以路段为图节点、路段之间的直接连通关系为边，通过多层图注意力网络聚合相邻路段的信息。最后，模型进一步融合路段长度、入度和出度等静态属性，得到路段嵌入。由此，路段表示既包含网格位置特征，也包含道路之间的拓扑联系。

对于输入轨迹中的每个观测点 p_i，图左下方的子图生成模块（Sub-Graph Generation）检索距离该点不超过 δ 的路段，并按照原路网中的连接关系构造局部道路子图 $G_i = (V_i, E_i)$。其中，V_i 为选中的路段集合，E_i 为这些路段之间的连接关系，δ 控制观测点的空间感受范围。模型根据观测点到各路段的距离分配权重，并通过加权平均得到该点的初始道路环境表示：

$$g_i = \frac{\sum_{e \in V_i} w_{i,e}\, x_e}{\sum_{e \in V_i} w_{i,e}}, \qquad w_{i,e} = \exp\left(-\frac{d(p_i, e)^2}{\gamma^2}\right)$$

其中，x_e 为 GridGNN 输出的路段嵌入，$d(p_i, e)$ 为观测点到路段投影位置的距离，$\gamma > 0$ 控制距离权重的衰减。模型将 g_i 与观测时间、网格坐标拼接并进行线性变换，形成序列输入；同时保留各点对应的子图结构及路段节点特征，供后续图细化使用。

图左上方的 GPSFormer 由 N 个 GPSFormerBlock 堆叠而成。每个 Block 先通过 Transformer 编码层，在不同观测点之间进行自注意力计算，得到包含前后观测信息的点级表示；再将这一表示送入对应的图细化层（Graph Refinement Layer）。图细化层通过门控融合，将当前点的序列上下文与其子图中各路段的节点特征结合，随后利用图注意力网络沿道路连接关系传播信息，更新局部道路表示。图中的 Graph Readout 再对更新后的子图节点进

行平均池化，得到新的点级表示，作为下一层 Transformer 的输入。

因此，GPSFormer 并非独立提取时间特征和空间特征后进行一次拼接，而是在各层中反复执行“序列上下文建模—道路子图更新—点级表示汇聚”。前后观测信息能够影响局部路段特征，更新后的路段特征又参与下一层序列编码，从而建立轨迹上下文与周围路网之间的联系。

图右上方的解码器沿用 MTrajRec 的多任务恢复思路。注意力模块根据当前解码状态，对 GPSFormer 输出的观测点表示进行加权；GRU 结合该上下文向量、上一时刻预测的路段嵌入与段内比例，逐步生成目标采样间隔下的恢复结果。每个恢复点仍表示为 $(\hat{e}_j, \hat{r}_j, t_j)$，分别对应路段编号、段内位置比例和时刻。图中的 Mask FFN 用于路段预测，另一条 FFN 分支根据预测路段及解码状态估计段内比例。约束掩码在已有观测对应的时刻依据距离调整路段概率，在缺失时刻则不施加这一直接距离限制。

训练时，除解码端的路段分类损失与比例回归损失外，模型还对 GPSFormer 最后一层输出的道路子图施加分类监督：根据子图节点特征及距离权重，预测各输入观测点对应的真实路段。这一辅助任务直接约束编码器的道路辨识能力，而非仅依靠最终恢复结果提供监督。联合损失为：

$$L_{\text{total}} = L_{\text{id}} + \lambda_1 L_{\text{rate}} + \lambda_2 L_{\text{enc}}$$

其中，L_{id} 为解码端路段分类的交叉熵损失，L_{rate} 为段内比例预测的均方误差，L_{enc} 为编码端的子图路段分类损失，λ_1 和 λ_2 用于平衡各项训练目标。

该方法的优势在于，将观测点周围的道路连接关系纳入编码，并通过多层交互利用前后观测辅助判断局部道路归属。但子图范围会影响模型可利用的道路信息，范围扩大也会增加图计算开销。此外，路网表示与分类监督能够引导道路选择，并不构成路段序列连通性的硬约束，恢复结果仍可能存在路段选择及段内定位误差。

（4）任务驱动型轨迹恢复方法

任务驱动型轨迹恢复方法围绕具体应用中的观测条件与恢复目标设计模型，将数据关联、缺失信息推断和路径生成等环节结合起来。本节以交通摄像头过车记录场景为例，介绍 TrajRecovery[49]。该方法面向缺少连续 GPS 记录的车辆，利用不同摄像头捕获的车辆快照恢复行驶路径。如图 3.23 所示，其框架由预处理器（Preprocessor）、空间转移概率模型（STPM）和轨迹生成器（Trajectory Generator）组成，分别负责整理车辆观测、估计道路转移概率及生成连接观测位置的路径。

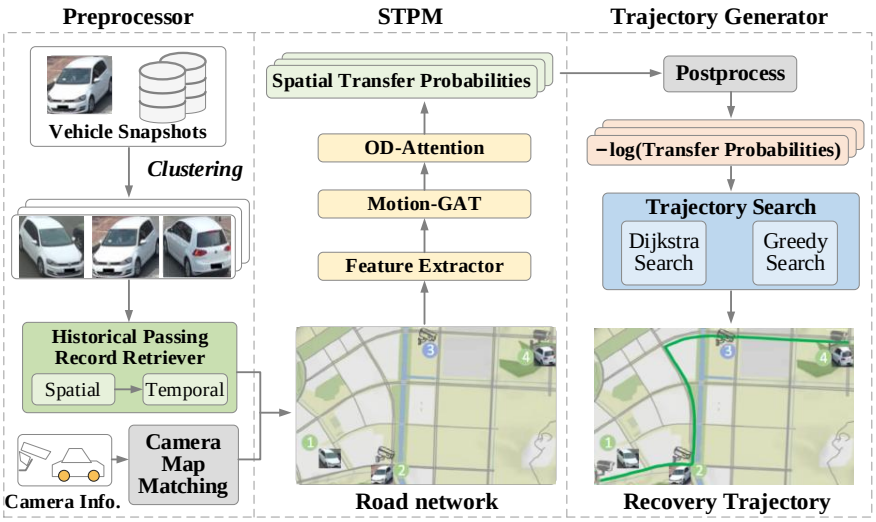


图 3.23 TrajRecovery 轨迹恢复系统框架（改绘自[49]）

图左侧的预处理器首先执行摄像头地图匹配。系统根据摄像头坐标和名称的可靠程度，分别采用邻近位置匹配、名称文本匹配或二者结合的策略确定交叉口，再利用拍摄方向关联对应路段。对于车辆快照，系统提取车辆外观与车牌特征，通过加权相似度进行聚类，将不同摄像头捕获的同一车辆记录按时间排列，得到稀疏过车序列。设车辆 p 的观测序列为 $J_p = \langle r_1, \ldots, r_n \rangle$，其中 $r_i = (c_i, t_i, e_i)$ 分别记录摄像头、过车时刻和对应路段。

针对漏拍造成的记录缺失，图中的历史过车记录检索器（Historical Passing Record Retriever）进一步进行空间与时间两级检索。空间检索利用当前观测前后的摄像头序列，与历史子轨迹计算最长公共子序列相似度，筛选相近的行驶片段；时间检索再比较相同摄像头之间的通行耗时，保留时间特征接近的历史片段。系统据此选取区间内高频出现的中间摄像头，并估计过车时间，补入推断记录。该过程利用出行重复性补充路径线索，但补入记录仍可能与本次实际行程存在差异。

图中部的 STPM 用于估计车辆从当前路段驶向各相邻路段的概率。底层的特征提取器（Feature Extractor）将道路类型、道路等级等类别属性转换为嵌入，将长度、宽度及距目标位置的距离等数值属性进行线性变换，再融合时间段嵌入，形成路段的时空表示。其中，时间信息按一天 48 个时段进行编码，以区分不同时段的出行条件。

随后，Motion-GAT 聚合周围路段的信息。该模块将左转、右转、直行和掉头，与路段间的转入、转出方向结合，形成不同的运动关系嵌入，并将其加入相邻路段特征。多头注意力再根据这些特征对邻接路段进行加权聚合，使模型能够结合道路条件及转向关系估计路径选择，而非仅记忆特定摄像头之间的转移频率。这一设计为摄像头未覆盖路段的概率估计提供了可利用的道路特征。

图中的 OD-Attention 进一步引入车辆历史选择。这里的 OD-pair 表示从路段 e_a 到相邻路段 e_b 的一次局部转移。模型将当前候选转移与该车辆的历史转移分别编码，通过注意力检索相似道路条件下的历史选择，并融合到当前转移表示中。最后，利用全连接网络计算各候选转移的得分，并在当前路段的可达后继集合 $A(e_a)$ 上归一化：

$$P_{ab} = \frac{\exp(s_{ab})}{\sum_{e_v \in A(e_a)} \exp(s_{av})}$$

其中，s_{ab} 为候选转移的网络输出得分。P_{ab} 表示给定目标路段、目标到达时刻、车辆身份及历史记录条件下，从 e_a 转移至 e_b 的概率。模型结合快照中的车道转向信息与路网构造训练样本，无需以高采样率 GPS 轨迹作为该概率模型的训练数据。

图右侧的轨迹生成器依次恢复相邻过车记录之间的道路路径。在模型采用的一阶马尔可夫假设下，路径概率可分解为各相邻路段转移概率的乘积。图中的“−log（Transfer Probabilities）”将概率转换为可累加的路径代价。设两个观测对应的起终路段为 e_s 和 e_d，合法连接路径集合为 $\Omega(e_s, e_d)$，对于路径 $R = \langle e_0, \ldots, e_K \rangle$，搜索目标为：

$$R^* = \arg \min_{R \in \Omega(e_s, e_d)} \sum_{k=0}^{K-1} \left[-\log P_{e_k, e_{k+1}} \right]$$

系统以路段为节点、路段间的连接关系为边构建搜索图，并采用 Dijkstra 算法寻找总代价最小的路径，再依次连接各观测区间的恢复结果。图中的 Greedy Search 对应论文的贪心对照版本，每一步选择概率最大的后继路段；Dijkstra 搜索则比较连接起终路段的累计代价，避免仅依据当前一步作出选择。这里的最优性针对模型给定的转移概率，并不意味着一定恢复了车辆的真实路线。

该方法的特点在于，将摄像头记录关联、道路条件建模与车辆历史偏好结合起来，为缺少连续 GPS 记录的车辆恢复行驶路径。但摄像头匹配错误、车辆聚类错误及历史记录补入偏差都可能影响后续结果。此外，其主要输出是连接过车观测的道路路径，并不直接给出每个缺失时刻的精确位置。

综合来看，传统轨迹恢复方法与深度学习轨迹恢复方法并不是简单的替代关系，而是适用于不同条件下的轨迹数据补全任务。传统方法具有较强的可解释性和较低的使用成本，适合作为基础方法或用于采样间隔较短、路网约束明确的场景；深度学习方法具有更强的表示能力和扩展潜力，适合处理低采样率、高噪声、复杂路网和多源感知数据下的轨迹恢复任务。未来的轨迹恢复研究将更加关注路网结构融合、个性化运动建模、可解释深度恢复、多源感知数据利用以及面向真实城市交通应用的高效恢复框架。

3.6 小结与展望

本章围绕轨迹质量评估、清洗、压缩与恢复四项任务，介绍了相应的问题定义、代表性方法及适用条件。这些任务分别回答数据存在哪些缺陷、哪些观测需要修正、如何减少表示规模，以及如何推断缺失信息等问题。由于轨迹记录具有时间顺序、空间关联和运动约束，治理过程既要检查字段缺失、重复与异常取值，也要考虑运动合理性、道路可达性及数据覆盖差异。

轨迹质量评估部分区分了任务无关评估与任务感知评估。前者依据数据自身的记录状态和约束条件检查质量，后者进一步分析质量问题对具体应用的影响。TsQuality、TFDV 和 F4U 分别提供了时间序列指标计算、机器学习数据验证及受控噪声实验等思路；T-Assess 则面向轨迹特征，将时间、空间、运动与分布覆盖等检查组织为专用评估框架。质量评估的输出不仅用于报告缺陷，还可为后续清洗、恢复及数据筛选确定处理范围与优先级。

轨迹清洗部分介绍了规则约束、平滑滤波、几何统计、混合约束及深度学习等方法。SCREEN 通过变化率约束与局部窗口修正观测，卡尔曼滤波结合运动预测与含噪观测估计状态，SHoTClean 则将硬约束和软评分纳入路径优化。深度方法通过生成概率、重构误差或插补误差识别可疑数据，并为修复提供候选值。需要区分的是，偏离常见路线或运动模式的行为不一定属于数据错误，是否修改仍需结合采集条件、运动约束与外部证据判断。

轨迹压缩部分区分了保留原始采样点的轨迹简化与低维潜在表示压缩。传统方法通过递归分裂、逐步合并或优化搜索，在保留点数与近似误差之间进行选择；深度方法则学习紧凑表示或关键点选择策略。PED、SED 和方向误差分别关注不同的信息偏差，重构损失又对应另一类信息保留目标。因此，比较方法时，需要同时说明输出形式、存储预算、误差度量及计算开销，不能仅依据保留点数或潜在维度判断压缩效果。

轨迹恢复部分区分了指定时刻的位置序列恢复与观测之间的道路路径恢复。几何插值利用局部运动假设估计位置，路网搜索与候选状态联合推断结合道路连接关系和观测匹配程度选择路径，深度方法进一步利用序列上下文、道路子图与历史行为辅助推断。交通摄像头场景还需要处理设备匹配、车辆关联和稀疏记录检索等问题。由于有限观测可能对应多条路径及多种速度变化过程，恢复结果应视为给定信息和模型条件下的估计，并根据输出类型评价位置误差或路径匹配程度。

表 3.1 归纳了本章四项治理任务的核心问题及代表方法。

表 3.1 轨迹大数据治理任务与代表方法

治理任务	核心问题	主要方法
质量评估	数据存在哪些缺陷，是否满足使用要求	TsQuality、TFDV、F4U、T-Assess
轨迹清洗	哪些已有观测需要修改，如何确定修正值	SCREEN、卡尔曼滤波、Akane、SHoTClean、GM-VSAE、TranAD、ImDiffusion
轨迹压缩	在预算或误差约束下如何缩减表示	DP、Hershberger–Snoeyink、Bottom-Up、SQUISH、Bellman、Error-Search、CVAE、RLTS、S3
轨迹恢复	根据有限观测如何推断缺失位置或路径	几何插值、Dijkstra、A*、HMM 地图匹配、ST-Matching、MTrajRec、ImIn-GAIL、RNTrajRec、TrajRecovery

四项任务之间存在反馈关系。质量评估可以确定需要处理的数据范围，清洗与恢复后需要重新检查约束和质量变化，压缩结果也应结合误差指标及应用效果进行验证。未来研究可重点关注以下方向。

（1）面向下游任务的治理效果评价

相同的数据缺陷对不同任务的影响并不一致。例如，短时缺失对起讫点统计的影响可能较小，却可能改变速度估计和驾驶行为识别结果。未来可将基础质量指标与下游任务表现结合，分析不同治理操作带来的收益与信息损失，并据此确定处理优先级。评价时还需保留独立验证数据，避免治理规则仅适应某个模型或某组测试样本。

（2）多源异构观测下的联合治理

GPS、摄像头过车记录、手机信令等数据在采样机制、误差形式和空间覆盖上存在差异。联合治理需要先解决对象身份、时间和空间位置的对应关系，再利用不同来源的证据交叉核验缺失与异常。对于观测冲突或部分来源缺失的情况，应记录各来源的可信程度及推断依据，避免将关联错误或推断结果直接当作真实观测传递到后续环节。

（3）运动约束、道路结构与不确定性表达

学习模型可以提供位置或路径候选，运动规律与道路约束则可用于检查候选的可行性。未来可进一步研究两者在训练和推断过程中的结合方式，并区分可容忍的观测偏差与必须满足的通行条件。对于稀疏观测不足以确定唯一结果的情况，可保留多条候选路径、置信评分或位置区间，使后续分析能够识别恢复结果中的不确定部分。

（4）轨迹流的动态治理与计算效率

持续到达的轨迹数据要求治理方法在有限缓存和响应时间内完成处理，同时适应采样策略、设备状态和道路条件的变化。未来可研究增量质量统计、局部修复、在线压缩及模型更新机制，减少对全部历史数据的重复计算。除准确性外，还应报告处理延迟、内存占用和更新成本，并说明在线结果在获得后续观测后如何修订。

（5）隐私保护与治理过程可追溯

轨迹清洗、恢复和多源关联可能改变或增加数据所揭示的出行信息，隐私保护需要贯穿数据接入、模型训练和结果使用。未来可结合访问控制、数据最小化及隐私保护学习等机制，明确不同使用目的下可保留的信息。同时，应记录数据来源、异常标记、修改位置、模型版本及修复依据，支持对治理结果的核查、回溯与撤销。

3.7 术语表

Trajectory Data	轨迹数据
Trajectory Point	轨迹点
Trajectory Data Quality Assessment	轨迹质量评估
Task-agnostic Quality Assessment	任务无关质量评估
Task-aware Quality Assessment	任务感知质量评估
Trajectory Cleaning	轨迹清洗
Duplicate Record	重复记录
Spatial Jump	空间跳跃
Anomaly Detection	异常检测
Map Matching	地图匹配
Trajectory Compression	轨迹压缩
Compression Ratio	压缩率
Key Point	关键点
Trajectory Simplification	轨迹简化
Direction Error	方向误差
Trajectory Recovery	轨迹恢复
Trajectory Imputation	轨迹插补
Low-sampling-rate Trajectory	低采样率轨迹

Linear Interpolation	线性插值

3.8 习题

习题 3.1 轨迹大数据治理主要包括轨迹质量评估、轨迹清洗、轨迹压缩和轨迹恢复四类基础任务。请分别说明这四类任务解决的核心问题，并简述它们在轨迹数据处理流程中的先后关系。

习题 3.2 轨迹质量评估可分为“任务无关的数据质量评估”和“任务感知的数据质量评估”。请说明二者的根本区别，并结合轨迹预测或交通状态识别任务解释为何仅依赖任务无关评估是不够的。

习题 3.3 与 TsQuality、TFDV、F4U 等通用数据质量评估方法相比，T-Assess 如何考虑轨迹数据的特有约束？请说明其四个核心质量维度，并解释“公平性”维度在轨迹数据治理中的意义。

习题 3.4 轨迹清洗首先需要判断“什么是异常”。请从物理约束、统计离群和地图匹配三个角度说明轨迹异常的常见判定标准，并各举一个例子。

习题 3.5 SHoTClean 是结合软硬约束的多变量时间序列清洗方法。请说明其“硬约束”和“软约束”分别起什么作用，并解释其为何可被转化为有向无环图中的最大权重路径搜索问题。

习题 3.6 TranAD 利用 Transformer 重构多变量时间序列并检测异常。请说明其两阶段机制，并设计将检测结果与重构值用于轨迹清洗的流程，解释候选修复、约束校验和效果评价各自的作用。

习题 3.7 Douglas-Peucker、SQUISH 和 Error-Search 分别代表三种不同思路的轨迹压缩方法。请比较它们的核心思想、适用场景与主要局限。

习题 3.8 RLTS 和 S3 均属于深度学习驱动的轨迹压缩方法。请说明它们各自将轨迹压缩建模为什么问题，并比较二者在“点选择机制”上的不同。

习题 3.9 线性插值、ST-Matching 和 MTrajRec 分别代表几何恢复、基于路网与时空约束的恢复、以及深度学习恢复三种思路。请比较它们的核心假设、优势与局限。

参考文献

[1] ZHENG Y. Trajectory data mining: An overview[J]. ACM Transactions on Intelligent

Systems and Technology, 2015, 6(3): 29:1-29:41.

[2] NEWSON P, KRUMM J. Hidden Markov map matching through noise and sparseness[C]//Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems. 2009: 336-343.

[3] ZHU J, WANG T, HU D, et al. T-Assess: An efficient data quality assessment system tailored for trajectory data[J]. Proceedings of the VLDB Endowment, 2024, 18(3): 666-674.

[4] WANG R Y, STRONG D M. Beyond accuracy: What data quality means to data consumers[J]. Journal of Management Information Systems, 1996, 12(4): 5-33.

[5] GPS.GOV. GPS accuracy[EB/OL]. [2026-11-08]. https://www.gps.gov/gps-accuracy.

[6] ZHUANG Z, KIM K H, SINGH J P. Improving energy efficiency of location sensing on smartphones[C]//Proceedings of the 8th International Conference on Mobile Systems, Applications, and Services. 2010: 315-330. DOI: 10.1145/1814433.1814464.

[7] LECORNU L, MONTAGNER J, PUENTES J. Reliability evaluation of incomplete AIS trajectories[C/OL]//COST MOVE Workshop on Moving Objects at Sea. Brest, 2013[2026-11-08]. https://hal.science/hal-01005502.

[8] CONG G, FAN W, GEERTS F, et al. Improving data quality: Consistency and accuracy[C]//Proceedings of the 33rd International Conference on Very Large Data Bases. 2007: 315-326.

[9] SCHELTER S, GRAFBERGER S, SCHMIDT P, et al. Differential data quality verification on partitioned data[C]//2019 IEEE 35th International Conference on Data Engineering. 2019: 1940-1945.

[10] FORONI D, LISSANDRINI M, VELEGRAKIS Y. Estimating the extent of the effects of data quality through observations[C]//2021 IEEE 37th International Conference on Data Engineering. 2021: 1913-1918.

[11] QIU Y, FANG C, SONG S, et al. TSQuality: Measuring time series data quality in Apache IoTDB[J]. Proceedings of the VLDB Endowment, 2023, 16(12): 3982-3985.

[12] CAVENESS E, GC P S, PENG Z, et al. TensorFlow data validation: Data analysis and validation in continuous ML pipelines[C]//Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data. 2020: 2793-2796.

[13] FORONI D, LISSANDRINI M, VELEGRAKIS Y. The F4U system for understanding the effects of data quality[C]//2021 IEEE 37th International Conference on Data Engineering. 2021: 2717-2720.

[14] GAO F, SONG S, WANG J. Time series data cleaning under multi-speed constraints[J]. International Journal of Software and Informatics, 2021, 11(1): 29-54.

[15] SONG S, GAO F, ZHANG A, et al. Stream data cleaning under speed and acceleration constraints[J]. ACM Transactions on Database Systems, 2021, 46(3): 10:1-10:44.

[16] SONG S, ZHANG A, WANG J, et al. SCREEN: Stream data cleaning under speed constraints[C]//Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data. 2015: 827-841.

[17] BRILLINGER D R. Time series: Data analysis and theory[M]. Philadelphia: Society for Industrial and Applied Mathematics, 2001.

[18] GARDNER E S. Exponential smoothing: The state of the art—Part II[J]. International Journal of Forecasting, 2006, 22(4): 637-666.

[19] KALMAN R E. A new approach to linear filtering and prediction problems[J]. Journal of Basic Engineering, 1960, 82(1): 35-45.

[20] YAMANISHI K, TAKEUCHI J. A unifying framework for detecting outliers and change points from non-stationary time series data[C]//Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. 2002: 676-681.

[21] HAN X, XIONG H, HE Z, et al. Akane: Perplexity-guided time series data cleaning[J]. Proceedings of the ACM on Management of Data, 2024, 2(3): 121.

[22] ZHANG A, SONG S, WANG J. Sequential data cleaning: A statistical approach[C]//Proceedings of the 2016 International Conference on Management of Data. 2016: 909-924.

[23] ZHANG A, WU Z, GONG Y, et al. Multivariate time series cleaning under speed constraints[J]. Proceedings of the ACM on Management of Data, 2024, 2(6): 245:1-245:26.

[24] DING X, LI G, WANG H, et al. Time series data cleaning under expressive constraints on both rows and columns[C]//2024 IEEE 40th International Conference on Data Engineering. 2024: 3682-3695.

[25] FANG Z, SHAO W, LU Z, et al. SHoTClean: Bridging soft and hard constraints for multivariate time series cleaning[J]. Proceedings of the ACM on Management of Data, 2026, 4(1): 84:1-84:27. DOI: 10.1145/3786698.

[26] LIU Y, ZHAO K, CONG G, et al. Online anomalous trajectory detection with deep generative sequence modeling[C]//2020 IEEE 36th International Conference on Data Engineering. 2020: 949-960.

[27] TULI S, CASALE G, JENNINGS N R. TranAD: Deep Transformer networks for anomaly detection in multivariate time series data[J]. Proceedings of the VLDB Endowment, 2022, 15(6): 1201-1214.

[28] CHEN Y, ZHANG C, MA M, et al. ImDiffusion: Imputed diffusion models for multivariate time series anomaly detection[J]. Proceedings of the VLDB Endowment, 2023, 17(3): 359-372.

[29] DOUGLAS D H, PEUCKER T K. Algorithms for the reduction of the number of points required to represent a digitized line or its caricature[J]. Cartographica, 1973, 10(2): 112-122.

[30] HERSHBERGER J, SNOEYINK J. Speeding up the Douglas-Peucker line-simplification algorithm: TR-92-07[R]. Vancouver: Department of Computer Science, University of British Columbia, 1992.

[31] KEOGH E, CHU S, HART D, et al. An online algorithm for segmenting time series[C]//Proceedings 2001 IEEE International Conference on Data Mining. 2001: 289-296.

[32] MUCKELL J, HWANG J H, PATIL V, et al. SQUISH: An online approach for GPS trajectory compression[C]//Proceedings of the 2nd International Conference on Computing for Geospatial Research & Applications. 2011: 1-8.

[33] BELLMAN R. On the approximation of curves by line segments using dynamic programming[J]. Communications of the ACM, 1961, 4(6): 284.

[34] LONG C, WONG R C W, JAGADISH H V. Trajectory simplification: On minimizing the direction-based error[J]. Proceedings of the VLDB Endowment, 2014, 8(1): 49-60.

[35] ELMAN J L. Finding structure in time[J]. Cognitive Science, 1990, 14(2): 179-211.

[36] SUTSKEVER I, VINYALS O, LE Q V. Sequence to sequence learning with neural

networks[C]//Advances in Neural Information Processing Systems 27. 2014: 3104-3112.

[37] RUMELHART D E, HINTON G E, WILLIAMS R J. Learning internal representations by error propagation[M]//RUMELHART D E, MCCLELLAND J L. Parallel distributed processing: Explorations in the microstructure of cognition. Vol. 1: Foundations. Cambridge, MA: MIT Press, 1986: 318-362.

[38] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[C]//Advances in Neural Information Processing Systems 30. 2017: 5998-6008.

[39] RÁKOS O, ARADI S, BÉCSI T, et al. Compression of vehicle trajectories with a variational autoencoder[J]. Applied Sciences, 2020, 10(19): 6739.

[40] WANG Z, LONG C, CONG G. Trajectory simplification with reinforcement learning[C]//2021 IEEE 37th International Conference on Data Engineering. 2021: 684-695.

[41] BELLMAN R. A Markovian decision process[J]. Journal of Mathematics and Mechanics, 1957, 6(5): 679-684.

[42] FANG Z, HE C, CHEN L, et al. A lightweight framework for fast trajectory simplification[C]//2023 IEEE 39th International Conference on Data Engineering. 2023: 2386-2399.

[43] DIJKSTRA E W. A note on two problems in connexion with graphs[J]. Numerische Mathematik, 1959, 1: 269-271.

[44] HART P E, NILSSON N J, RAPHAEL B. A formal basis for the heuristic determination of minimum cost paths[J]. IEEE Transactions on Systems Science and Cybernetics, 1968, 4(2): 100-107.

[45] LOU Y, ZHANG C, ZHENG Y, et al. Map-matching for low-sampling-rate GPS trajectories[C]//Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems. 2009: 352-361.

[46] REN H, RUAN S, LI Y, et al. MTrajRec: Map-constrained trajectory recovery via Seq2Seq multi-task learning[C]//Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining. 2021: 1410-1419.

[47] CHEN Y, ZHANG H, SUN W, et al. RNTrajRec: Road network enhanced trajectory recovery with spatial-temporal Transformer[C]//2023 IEEE 39th International Conference

on Data Engineering. 2023: 829-842.

[48] WEI H, CHEN C, LIU C, et al. Learning to simulate on sparse trajectory data[C]//Machine Learning and Knowledge Discovery in Databases. Cham: Springer, 2020: 530-545.

[49] WU D, FANG Z, SUN Q, et al. TrajRecovery: An efficient vehicle trajectory recovery framework based on urban-scale traffic camera records[C]//Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. 2024: 5979-5990.